In this lesson, you can find the full source code for the project used within the course. Feel free to use this lesson as needed – whether to help you debug your code, use it as a reference later, or expand the project to practice your skills!

AGENTS.md

Found in folder: **/**

This file provides setup instructions and conventions for a freelancer invoice tracking application built with Next.js. It outlines the database schema, data layer rules, page structure, and coding standards for the project. The document serves as a reference for developers working on the app.

<!-- BEGIN:nextjs-agent-rules -->

This is NOT the Next.js you know

This version has breaking changes — APIs, conventions, and file structure may all differ from your training data. Read the relevant guide **in** `node_modules/next/dist/docs/` before writing any code. Heed deprecation notices.

<!-- **END**:nextjs-agent-rules -->

Freelancer Invoice Tracker

A single-user freelancer bookkeeping app **for** managing clients, projects, invoices, and invoice line items.

Tech Stack

- Next.js 16 App Router

- React 19

- JavaScript

- Tailwind CSS 4

- SQLite

- better-sqlite3

- ESLint 9 with `eslint-config-next`

- npm

Key Commands

- Dev server: `npm run dev`

- Tests: `npm run test`

- Lint: `npm run lint`

- Database migration: `node src/lib/migrate.js`

- Database seed: `node src/lib/seed.js`

Database Schema

Use SQLite with foreign keys enabled. Store money as integer cents and dates/timestamps as ISO text.

`clients`

- Stores freelancer clients and contact details.

- Key columns: `id`, `name`, `email`, `phone`, `company`, `address`, `notes`, `created_at`, `updated_at`.

- Constraints: `name` is required.

- Index: client name.

`projects`

- Stores projects owned by clients.

- Key columns: `id`, `client_id`, `name`, `description`, `status`, `hourly_rate_cents`, `created_at`, `updated_at`.

- Relationship: `client_id` references `clients(id)` with restricted deletion.

- Constraints: status is one **of** `active`, `paused`, `completed`, `archived`; hourly rate must be **null** or non-negative.

- Indexes: client, status.

`invoices`

- Stores invoices **for** projects.

- Key columns: `id`, `project_id`, `invoice_number`, `status`, `issue_date`, `due_date`, `currency`, `subtotal_cents`, `total_cents`, `paid_cents`, `sent_at`, `paid_at`, `notes`, `created_at`, `updated_at`.

- Relationship: `project_id` references `projects(id)` with restricted deletion.

- Constraints: invoice number is unique; status is one **of** `draft`, `sent`, `paid`, `overdue`, `**void**`; totals and paid amounts are non-negative; paid amount cannot exceed total; currency is a 3-letter code.

- Indexes: project, status, due date, issue date.

`invoice_line_items`

- Stores ordered line items **for** invoices.

- Key columns: `id`, `invoice_id`, `position`, `description`, `quantity`, `unit_price_cents`, `line_total_cents`, `created_at`, `updated_at`.

- Relationship: `invoice_id` references `invoices**(**id**)**` with cascade deletion.

- Constraints: position is non-negative; quantity is positive; unit price and line total are non-negative; each invoice has unique line item positions.

- Index: invoice.

Invoice totals are recalculated server-side whenever line items change. **In** v1, `subtotal_cents` and `total_cents` are equal because tax and discounts are out **of** scope.

Data Layer

- All database access goes through `src/lib/services.js`.

- Never write raw SQL queries inside components or pages.

- All money values are stored and passed as integer cents.

- Run `npm run test` before completing any task that touches the service layer.

Server Actions

- All data mutations go through Server Actions **in** `src/actions/`.

- Actions import from `src/lib/services.js` only; never query the database directly.

- Every action returns `**{** data, error **}**`; `error` is a string message or **null**.

- Action files must start with `'use server'`.

- **For** reads, use service functions directly **in** Server Components; no action needed.

- Never use `fetch**()**` to call our own data; use actions or service functions directly.

Page Structure

Use `src/app` routes with shared navigation **for** Dashboard, Clients, Projects, and Invoices.

- `/`: dashboard with unpaid total, overdue invoices, paid **this** month, active projects, recent invoices, upcoming due invoices, and links to create core records.

- `/clients`: client list with search/filter, project counts, unpaid totals, and links to detail/create pages.

- `/clients/**new**`: create client form.

- `/clients/**[**clientId**]**`: client detail with contact info, notes, projects, and invoices across the client's projects.

- `/clients/**[**clientId**]**/edit`: edit client form.

- `/projects`: project list with status/client filters, invoice counts, unpaid totals, and links to detail/create pages.

- `/projects/**new**`: create project form.

- `/projects/**[**projectId**]**`: project detail with client link, invoices, and totals by status.

- `/projects/**[**projectId**]**/edit`: edit project form.

- `/invoices`: invoice list with status, client/project, due date, and invoice number filtering.

- `/invoices/**new**`: create invoice form with project selection and editable line items.

- `/invoices/**[**invoiceId**]**`: invoice detail and print-friendly invoice view with actions to edit, mark sent, mark paid, or **void**.

- `/invoices/**[**invoiceId**]**/edit`: edit invoice metadata and line items.

Conventions

- Read the relevant guide **in** `node_modules/next/dist/docs/` before changing App Router code.

- Keep routes under `src/app`.

- Use `page.js` **for** route UI and `layout.js` **for** shared route shells.

- Use dynamic route segment folders with bracket names, such as `**[**clientId**]**`, `**[**projectId**]**`, and `**[**invoiceId**]**`.

- Use lowercase kebab-case folder names **for** static route segments.

- Keep shared UI components outside route folders when reused across multiple sections.

- Keep route-specific components close to the route that owns them.

- Use Server Components by default.

- Use Client Components only **for** interactive browser state.

- Put server-only database access behind a dedicated data layer instead **of** querying SQLite directly from UI components.

- Use Server Functions **for** form mutations.

- Recalculate derived invoice totals on the server, not from trusted client input.

- Use integer cents **for** all money values.

- Use ISO text values **for** dates and timestamps.

- Name database columns **in** snake_case.

- Name JavaScript variables and functions **in** camelCase.

- Name React components **in** PascalCase.

Shared Components

- Use `StatusBadge` at `src/components/StatusBadge.jsx` **for** all status display. Never hardcode status colours inline.

- Use `formatCurrency`, `formatDate`, and `formatInvoiceNumber` at `src/lib/formatters.js` **for** all currency, date, and invoice number formatting. Never format these values inline.

Critical Rules

- NEVER run npm run build — it hangs **in this** environment. Use npm run test **for** the test suite only.

- **In** InvoiceForm, quantity inputs must have step="1" and min="1" — never leave number inputs without explicit step and min attributes.

RevenueChart.jsx

Found in folder: **/src/app**

This code renders a bar chart displaying revenue data over months, with currency-formatted axes and tooltips. It uses a custom formatter to abbreviate large values on the Y-

axis for better readability. The chart is styled responsively within a container and includes grid lines, axis labels, and interactive tooltips.

```
"use client";

import {
  Bar,
  BarChart,
  CartesianGrid,
  ResponsiveContainer,
  Tooltip,
  XAxis,
  YAxis,
} from "recharts";

import { formatCurrency } from "@/lib/formatters";

import styles from "./Dashboard.module.css";

// Abbreviates large currency values for Y-axis display (e.g., 100000 cents becomes "$1k")
function formatAxisCurrency(cents) {
  if (cents >= 100000) {
    return `$${Math.round(cents / 100000)}k`;
  }
  return formatCurrency(cents).replace(".00", ""); // Remove cents when zero for cleaner labels
}

export default function RevenueChart({ data }) {
  return (
    <div className={styles.chartFrame}>
      <ResponsiveContainer width="100%" height="100%">
        <BarChart data={data} margin={{ top: 8, right: 8, left: 0, bottom: 0 }}>
```

```
<CartesianGrid stroke="#e2e8f0" vertical={false} />
<XAxis
 dataKey="month" // Expects data objects with a "month" property
 axisLine={false}
 tickLine={false}
 tick={{ fill: "#64748b", fontSize: 12 }}
/>
<YAxis
 axisLine={false}
 tickLine={false}
 tick={{ fill: "#64748b", fontSize: 12 }}
 tickFormatter={formatAxisCurrency} // Applies custom formatting to Y-axis labels
 width={64} // Reserved width for formatted tick labels
/>
<Tooltip
 cursor={{ fill: "#f1f5f9" }}
 formatter={((value) => [formatCurrency(value), "Revenue"]} // Formats tooltip values as currency
 labelStyle={{ color: "#172033", fontWeight: 700 }}
 contentStyle={{
  border: "1px solid #d9e1ec",
  borderRadius: 8,
  boxShadow: "0 12px 28px rgba(15, 23, 42, 0.12)",
 }}
/>
<Bar dataKey="totalCents" fill="#2563eb" radius={[6, 6, 0, 0]} /> // Maps "totalCents" from data to bar heights
```

```
      </BarChart>
    </ResponsiveContainer>
  </div>
 );
}
```

ClientInlineForm.jsx

Found in folder: **/src/app/clients**

This code provides an inline form component for adding or editing client information. It conditionally renders either a button to open the form or the form itself based on state, with different behaviors for create versus edit modes. When submitted, it calls server actions to create or update client data and handles loading states and errors.

```
"use client";

import { useRouter } from "next/navigation";

import { useState, useTransition } from "react";

import {
  createClientAction,
  updateClientAction,
} from "@/actions/clientActions";

import styles from "./ClientsPage.module.css";

// Default empty values for new client form

const emptyFormData = {
  name: "",
  company: "",
  email: "",
  phone: "",
  address: "",
  notes: "",
```

```
};
// Convert client object to form data, handling missing properties
function clientToFormData(client) {
  return {
    name: client?.name || "",
    company: client?.company || "",
    email: client?.email || "",
    phone: client?.phone || "",
    address: client?.address || "",
    notes: client?.notes || "",
  };
}
export default function ClientInlineForm({ mode, client = null }) {
  const router = useRouter();
  const [isOpen, setIsOpen] = useState(false);
  // Initialize form with client data for edit mode or empty values for create
  const [formData, setFormData] = useState(() => (
    mode === "edit" ? clientToFormData(client) : emptyFormData
  ));
  const [error, setError] = useState(null);
  const [isPending, startTransition] = useTransition();
  const isEdit = mode === "edit";
  function openForm() {
    setFormData(isEdit ? clientToFormData(client) : emptyFormData);
    setError(null);
    setIsOpen(true);
```

```
}
function closeForm() {
  setFormData(isEdit ? clientToFormData(client) : emptyFormData);
  setError(null);
  setIsOpen(false);
}
function updateField(event) {
  const { name, value } = event.target;
  setFormData((current) => ({
    ...current,
    [name]: value,
  }));
}
function submitForm(event) {
  event.preventDefault();
  setError(null);
  // Use transition for loading state during async server action
  startTransition(async () => {
    const result = isEdit
      ? await updateClientAction(client.id, formData)
      : await createClientAction(formData);
    if (result.error) {
      setError(result.error);
      return;
    }
    // Reset form and refresh page data on success
```

```
    setIsOpen(false);
    setFormData(emptyFormData);
    router.refresh();
  });
}
// Show button to open form when closed
if (!isOpen) {
  return (
    <button
      className={isEdit ? styles.secondaryButton : styles.primaryButton}
      type="button"
      onClick={openForm}
    >
      {isEdit ? "Edit" : "Add Client"}
    </button>
  );
}
// Render form when open
return (
  <form
    className={`${styles.inlineForm} ${isEdit ? styles.editForm : ""}`}
    onSubmit={submitForm}
  >
    <div className={styles.formGrid}>
      <label className={styles.field}>
        <span>Name</span>
```

```
 <input
  name="name"
  required
  value={formData.name}
  onChange={updateField}
 />
</label>
<label className={styles.field}>
 <span>Company</span>
 <input
  name="company"
  value={formData.company}
  onChange={updateField}
 />
</label>
<label className={styles.field}>
 <span>Email</span>
 <input
  name="email"
  type="email"
  value={formData.email}
  onChange={updateField}
 />
</label>
<label className={styles.field}>
 <span>Phone</span>
```

```
          <input
            name="phone"
            value={formData.phone}
            onChange={updateField}
          />
        </label>
        <label className={`${styles.field} ${styles.fullWidth}`}>
          <span>Address</span>
          <textarea
            name="address"
            rows="2"
            value={formData.address}
            onChange={updateField}
          />
        </label>
        <label className={`${styles.field} ${styles.fullWidth}`}>
          <span>Notes</span>
          <textarea
            name="notes"
            rows="2"
            value={formData.notes}
            onChange={updateField}
          />
        </label>
      </div>
      {error ? <p className={styles.error}>{error}</p> : null}
```

```
    <div className={styles.formActions}>
     <button
      className={styles.primaryButton}
      type="submit"
      disabled={isPending}
     >
      {isPending ? "Saving..." : isEdit ? "Save" : "Create"}
     </button>
     <button
      className={styles.secondaryButton}
      type="button"
      onClick={closeForm}
      disabled={isPending}
     >
      Cancel
     </button>
    </div>
   </form>
  );
}
```

DeleteConfirmModal.jsx

Found in folder: **/src/app/invoices**

This component renders a modal dialog that asks the user to confirm the deletion of a specific invoice. It calls a delete action and handles the pending state and any errors that may occur. Upon successful deletion, it closes the modal and refreshes the page data.

```
"use client";

import { useRouter } from "next/navigation";
```

```
import { useState, useTransition } from "react";
import { deleteInvoiceAction } from "@/actions/invoiceActions";
import { formatInvoiceNumber } from "@/lib/formatters";
import styles from "./DeleteConfirmModal.module.css";
// Safely extracts the invoice number from various possible property names
function getInvoiceNumber(invoice) {
 return invoice?.invoice_number ?? invoice?.invoiceNumber ?? invoice?.id ?? "";
}
export default function DeleteConfirmModal({ invoice, onClose }) {
 const router = useRouter();
 const [error, setError] = useState(null);
 const [isPending, startTransition] = useTransition();
 const invoiceNumber = formatInvoiceNumber(getInvoiceNumber(invoice));
 function confirmDelete() {
  setError(null);
  startTransition(async () => {
   const result = await deleteInvoiceAction(invoice.id);
   if (result.error) {
    setError(result.error);
    return;
   }
   onClose();
   router.refresh(); // Refresh server components after successful deletion
  });
 }
 return (
```

```
<div className={styles.backdrop} role="presentation">
 <div
  aria-labelledby="delete-invoice-title"
  aria-modal="true"
  className={styles.modal}
  role="dialog"
 >
  <h2 className={styles.title} id="delete-invoice-title">
   Delete invoice
  </h2>
  <p className={styles.message}>
   Are you sure you want to delete invoice {invoiceNumber}?
  </p>
  {error ? <p className={styles.error}>{error}</p> : null}
  <div className={styles.actions}>
   <button
    className={styles.dangerButton}
    type="button"
    onClick={confirmDelete}
    disabled={isPending}
   >
    {isPending ? "Deleting..." : "Confirm"}
   </button>
   <button
    className={styles.secondaryButton}
    type="button"
```

```
          onClick={onClose}
          disabled={isPending}
        >
          Cancel
        </button>
      </div>
    </div>
  </div>
 );
}
```

InvoiceActions.jsx

Found in folder: **/src/app/invoices**

This component renders action buttons for an invoice, including View, Edit, Mark as Paid, and Delete. It conditionally shows the Mark as Paid button only if the invoice isn't already paid or void. It handles marking an invoice as paid with a transition state and error handling, and opens a delete confirmation modal.

```
"use client";

import Link from "next/link";

import { useRouter } from "next/navigation";

import { useState, useTransition } from "react";

import { updateInvoiceStatusAction } from "@/actions/invoiceActions";

import DeleteConfirmModal from "./DeleteConfirmModal";

import styles from "./InvoiceActions.module.css";

export default function InvoiceActions({ invoice }) {
  const router = useRouter();
  const [isDeleteOpen, setIsDeleteOpen] = useState(false);
  const [error, setError] = useState(null);
```

```
const [isPending, startTransition] = useTransition();
if (!invoice) {
  return null;
}
// Only allow marking as paid if invoice is not already paid or void
const canMarkPaid = invoice.status !== "paid" && invoice.status !== "void";
function markAsPaid() {
  setError(null);
  startTransition(async () => {
    const result = await updateInvoiceStatusAction(invoice.id, "paid");
    if (result.error) {
      setError(result.error);
      return;
    }
    router.refresh();
  });
}
return (
  <>
    <div className={styles.actions}>
      <Link
        className={styles.tertiaryButton}
        href={`/invoices/${invoice.id}`}
      >
        View
      </Link>
```

```
<Link
  className={styles.secondaryButton}
  href={`/invoices/${invoice.id}/edit`}
>
  Edit
</Link>
{canMarkPaid ? (
  <button
    className={styles.primaryButton}
    type="button"
    onClick={markAsPaid}
    disabled={isPending}
  >
    {isPending ? "Updating..." : "Mark as Paid"}
  </button>
) : null}
<button
  className={styles.dangerButton}
  type="button"
  onClick={() => {
    setError(null);
    setIsDeleteOpen(true);
  }}
  disabled={isPending}
>
  Delete
```

```
      </button>
    </div>
    {error ? <p className={styles.error}>{error}</p> : null}
    {isDeleteOpen ? (
      <DeleteConfirmModal
        invoice={invoice}
        onClose={() => setIsDeleteOpen(false)}
      />
    ) : null}
  </>
 );
}
```

InvoiceForm.jsx

Found in folder: **/src/app/invoices**

This code provides a form for creating or editing an invoice. It manages form state for invoice details and a list of line items, including calculations for totals. It validates input, builds a payload, and submits it to create or update an invoice via server actions.

```
"use client";

import { useMemo, useState, useTransition } from "react";

import { useRouter } from "next/navigation";

import {
  createInvoiceAction,
  updateInvoiceAction,
} from "@/actions/invoiceActions";

import { formatCurrency } from "@/lib/formatters";

import styles from "./InvoiceForm.module.css";

const invoiceStatuses = ["draft", "sent", "paid", "overdue", "void"];
```

```
const emptyLineItem = {
  description: "",
  quantity: "1",
  unitPriceDollars: "",
};
// Generates a unique key for each line item
function nextLineItemKey() {
  if (typeof crypto !== "undefined" && crypto.randomUUID) {
    return crypto.randomUUID();
  }
  return `${Date.now()}-${Math.random()}`;
}
function createLineItem(overrides = {}) {
  return {
    key: nextLineItemKey(),
    ...emptyLineItem,
    ...overrides,
  };
}
// Converts cents (stored in DB) to a dollar string for display
function centsToDollars(cents) {
  if (cents === undefined || cents === null || cents === "") {
    return "";
  }
  return (Number(cents) / 100).toFixed(2);
}
```

```
// Converts user-entered dollar string to cents for storage
function dollarsToCents(value) {
  const text = String(value || "").trim();
  if (text === "") {
    return 0;
  }
  const normalizedText = text.replace(/[$£€,\s]/g, "");
  if (!/^\d+(\.\d{1,2})?$/.test(normalizedText)) {
    return Number.NaN;
  }
  return Math.round(Number(normalizedText) * 100);
}
function getTodayDate() {
  return new Date().toISOString().slice(0, 10);
}
// Determine initial client selection from invoice data or client list
function getInitialClientId(invoice, clients) {
  if (invoice?.client?.id) {
    return String(invoice.client.id);
  }
  return clients[0]?.id ? String(clients[0].id) : "";
}
// Determine initial project selection, filtered by selected client
function getInitialProjectId(invoice, projects, clientId) {
  if (invoice?.projectId) {
    return String(invoice.projectId);
```

```
  }
  const firstProject = projects.find(
    (project) => String(project.clientId) === String(clientId),
  );
  return firstProject?.id ? String(firstProject.id) : "";
}
// Converts invoice data from server into the form's line item structure
function invoiceToLineItems(invoice) {
  if (!invoice?.lineItems?.length) {
    return [createLineItem()];
  }
  return invoice.lineItems.map((item) =>
    createLineItem({
      description: item.description || "",
      quantity: String(item.quantity ?? "1"),
      unitPriceDollars: centsToDollars(item.unitPriceCents),
    }),
  );
}
// Calculates the total cost for a single line item in cents
function getLineTotalCents(item) {
  const quantity = Number(item.quantity || 0);
  const unitPriceCents = dollarsToCents(item.unitPriceDollars);
  if (!Number.isFinite(quantity) || Number.isNaN(unitPriceCents)) {
    return 0;
  }
```

```
  return Math.round(quantity * unitPriceCents);
}
export default function InvoiceForm({
  mode,
  invoice = null,
  clients,
  projects,
}) {
  const router = useRouter();
  const isEdit = mode === "edit";
  const initialClientId = getInitialClientId(invoice, clients);
  const [selectedClientId, setSelectedClientId] = useState(initialClientId);
  const [selectedProjectId, setSelectedProjectId] = useState(() =>
    getInitialProjectId(invoice, projects, initialClientId),
  );
  const [invoiceNumber, setInvoiceNumber] = useState(
    invoice?.invoiceNumber || "",
  );
  const [issueDate, setIssueDate] = useState(
    invoice?.issueDate || getTodayDate(),
  );
  const [dueDate, setDueDate] = useState(invoice?.dueDate || "");
  const [status, setStatus] = useState(invoice?.status || "draft");
  const [lineItems, setLineItems] = useState(() => invoiceToLineItems(invoice));
  const [error, setError] = useState(null);
  const [isPending, startTransition] = useTransition();
```

```
// Filter projects based on the currently selected client
const availableProjects = useMemo(
  () =>
    projects.filter(
      (project) => String(project.clientId) === String(selectedClientId),
    ),
  [projects, selectedClientId],
);
// Calculate the total invoice amount from all line items
const totalCents = useMemo(
  () =>
    lineItems.reduce((sum, item) => sum + getLineTotalCents(item), 0),
  [lineItems],
);
const hasClients = clients.length > 0;
const hasProjects = availableProjects.length > 0;
// Updates client and resets project selection if needed
function updateClient(event) {
  const nextClientId = event.target.value;
  const nextProjects = projects.filter(
    (project) => String(project.clientId) === String(nextClientId),
  );
  const selectedProjectStillValid = nextProjects.some(
    (project) => String(project.id) === String(selectedProjectId),
  );
  setSelectedClientId(nextClientId);
```

```
  setSelectedProjectId(
    selectedProjectStillValid ? selectedProjectId : String(nextProjects[0]?.id || ""),
  );
}
function updateLineItem(key, field, value) {
  setLineItems((current) =>
    current.map((item) =>
      item.key === key
        ? {
            ...item,
            [field]: value,
          }
        : item,
    ),
  );
}
function addLineItem() {
  setLineItems((current) => [...current, createLineItem()]);
}
// Ensures at least one line item remains after removal
function removeLineItem(key) {
  setLineItems((current) => {
    if (current.length === 1) {
      return [createLineItem()];
    }
    return current.filter((item) => item.key !== key);
```

```
  });
}
// Builds the data payload for the server action, validating line items
function buildPayload() {
  const parsedLineItems = lineItems.map((item, index) => {
    const unitPriceCents = dollarsToCents(item.unitPriceDollars);
    const quantity = Number(item.quantity);
    if (Number.isNaN(unitPriceCents)) {
      throw new Error(`Line item ${index + 1} unit price must be a valid dollar amount.`);
    }
    return {
      description: item.description,
      quantity,
      unitPriceCents,
    };
  });
  return {
    projectId: selectedProjectId,
    invoiceNumber,
    status,
    issueDate,
    dueDate,
    currency: invoice?.currency || "USD",
    paidCents: status === "paid" ? totalCents : 0,
    lineItems: parsedLineItems,
  };
```

```
}
function submitForm(event) {
 event.preventDefault();
 setError(null);
 let payload;
 try {
  payload = buildPayload();
 } catch (submissionError) {
  setError(submissionError.message);
  return;
 }
 startTransition(async () => {
  const result = isEdit
   ? await updateInvoiceAction(invoice.id, payload)
   : await createInvoiceAction(payload);
  if (result.error) {
   setError(result.error);
   return;
  }
  router.push("/invoices");
  router.refresh();
 });
}
function cancelForm() {
 router.push("/invoices");
}
```

```
return (
 <form className={styles.form} onSubmit={submitForm}>
  <div className={styles.panel}>
   <div className={styles.sectionHeader}>
    <div>
     <p className={styles.eyebrow}>Invoice</p>
     <h2 className={styles.sectionTitle}>Details</h2>
    </div>
   </div>
   <div className={styles.formGrid}>
    <label className={styles.field}>
     <span>Client</span>
     <select
      value={selectedClientId}
      onChange={updateClient}
      disabled={!hasClients || isPending}
      required
     >
      {hasClients ? null : <option value="">Add a client first</option>}
      {clients.map((client) => (
       <option key={client.id} value={client.id}>
        {client.name}
       </option>
      ))}
     </select>
    </label>
```

```
<label className={styles.field}>
 <span>Project</span>
 <select
  value={selectedProjectId}
  onChange={(event) => setSelectedProjectId(event.target.value)}
  disabled={!hasProjects || isPending}
  required
 >
  {hasProjects ? null : (
   <option value="">No projects for this client</option>
  )}
  {availableProjects.map((project) => (
   <option key={project.id} value={project.id}>
    {project.name}
   </option>
  ))}
 </select>
</label>
<label className={styles.field}>
 <span>Invoice number</span>
 <input
  value={invoiceNumber}
  onChange={(event) => setInvoiceNumber(event.target.value)}
  disabled={isPending}
  required
 />
```

```
</label>
<label className={styles.field}>
 <span>Status</span>
 <select
  value={status}
  onChange={(event) => setStatus(event.target.value)}
  disabled={isPending}
 >
  {invoiceStatuses.map((invoiceStatus) => (
   <option key={invoiceStatus} value={invoiceStatus}>
    {invoiceStatus}
   </option>
  ))}
 </select>
</label>
<label className={styles.field}>
 <span>Issue date</span>
 <input
  type="date"
  value={issueDate}
  onChange={(event) => setIssueDate(event.target.value)}
  disabled={isPending}
  required
 />
</label>
<label className={styles.field}>
```

```
    <span>Due date</span>
    <input
     type="date"
     value={dueDate}
     onChange={(event) => setDueDate(event.target.value)}
     disabled={isPending}
    />
   </label>
  </div>
 </div>
 <div className={styles.panel}>
  <div className={styles.sectionHeader}>
   <div>
    <p className={styles.eyebrow}>Line items</p>
    <h2 className={styles.sectionTitle}>Billable work</h2>
   </div>
   <button
    className={styles.secondaryButton}
    type="button"
    onClick={addLineItem}
    disabled={isPending}
   >
    Add Line Item
   </button>
  </div>
  <div className={styles.lineItems}>
```

```
<div className={styles.lineItemHeader} aria-hidden="true">
  <span>Description</span>
  <span>Quantity</span>
  <span>Unit price</span>
  <span>Line total</span>
  <span>Actions</span>
</div>
{lineItems.map((item, index) => (
  <div className={styles.lineItemRow} key={item.key}>
    <label className={styles.lineField}>
      <span>Description</span>
      <input
        value={item.description}
        onChange={(event) =>
          updateLineItem(item.key, "description", event.target.value)
        }
        placeholder={`Line item ${index + 1}`}
        disabled={isPending}
        required
      />
    </label>
    <label className={styles.lineField}>
      <span>Quantity</span>
      <input
        type="number"
        min="1"
```

```
    step="1"
    value={item.quantity}
    onChange={(event) =>
      updateLineItem(item.key, "quantity", event.target.value)
    }
    disabled={isPending}
    required
   />
  </label>
  <label className={styles.lineField}>
   <span>Unit price</span>
   <input
    inputMode="decimal"
    min="0"
    placeholder="125.00"
    step="0.01"
    type="number"
    value={item.unitPriceDollars}
    onChange={(event) =>
     updateLineItem(
      item.key,
      "unitPriceDollars",
      event.target.value,
     )
    }
    disabled={isPending}
```

```
          required
        />
      </label>
      <output className={styles.lineTotal}>
        <span>Line total</span>
        <strong>{formatCurrency(getLineTotalCents(item), "USD")}</strong>
      </output>
      <button
        className={styles.dangerButton}
        type="button"
        onClick={() => removeLineItem(item.key)}
        disabled={isPending}
      >
        Remove
      </button>
    </div>
  ))}
 </div>
 <div className={styles.totalRow}>
  <span>Total</span>
  <strong>{formatCurrency(totalCents, "USD")}</strong>
 </div>
</div>
{error ? <p className={styles.error}>{error}</p> : null}
<div className={styles.formActions}>
 <button
```

```
        className={styles.primaryButton}
        type="submit"
        disabled={isPending || !hasClients || !hasProjects}
      >
        {isPending ? "Saving..." : isEdit ? "Save Invoice" : "Create Invoice"}
      </button>
      <button
        className={styles.secondaryButton}
        type="button"
        onClick={cancelForm}
        disabled={isPending}
      >
        Cancel
      </button>
    </div>
  </form>
 );
}
```

InvoicesTable.jsx

Found in folder: **/src/app/invoices**

This code renders a table of invoices with filtering, sorting, and search capabilities. It allows users to filter by status, search by client or invoice number, and sort by due date or total amount. The component also provides options to export filtered data and clear applied filters.

```
"use client";

import Link from "next/link";

import { useMemo, useState } from "react";
```

```
import StatusBadge from "@/components/StatusBadge";
import {
  formatCurrency,
  formatDate,
  formatInvoiceNumber,
} from "@/lib/formatters";
import InvoiceActions from "./InvoiceActions";
import styles from "./InvoicesPage.module.css";
// Available status options for filtering invoices
const statusFilters = [
  { label: "All", value: "all" },
  { label: "Draft", value: "draft" },
  { label: "Sent", value: "sent" },
  { label: "Paid", value: "paid" },
  { label: "Overdue", value: "overdue" },
];
// Normalizes search input for consistent comparison
function normalizeSearch(value) {
  return String(value || "").trim().toLowerCase();
}
// Sorts invoices by total amount or due date in ascending/descending order
function sortInvoices(invoices, sortBy, sortDirection) {
  return [...invoices].sort((first, second) => {
    const direction = sortDirection === "asc" ? 1 : -1;
    if (sortBy === "total") {
      return (first.totalCents - second.totalCents) * direction;
```

```
    }
    const firstDue = first.dueDate || "";
    const secondDue = second.dueDate || "";
    return firstDue.localeCompare(secondDue) * direction;
  });
}
export default function InvoicesTable({
  hasInvoices,
  invoices,
  initialFilters = {},
}) {
  // Initialize status filter from server-side props if valid, otherwise default to "all"
  const initialStatus = statusFilters.some(
    (filter) => filter.value === initialFilters.status,
  )
    ? initialFilters.status
    : "all";
  const [statusFilter, setStatusFilter] = useState(initialStatus);
  const [searchTerm, setSearchTerm] = useState("");
  const [sortBy, setSortBy] = useState("dueDate");
  const [sortDirection, setSortDirection] = useState("asc");
  const hasServerFilters = Boolean(
    initialFilters.status || initialFilters.clientId || initialFilters.projectId,
  );
  function clearClientFilters() {
    setStatusFilter("all");
```

```
  setSearchTerm("");
}
// Generate export URL with current filters applied
const exportHref = useMemo(() => {
  const params = new URLSearchParams();
  if (statusFilter !== "all") {
    params.set("status", statusFilter);
  }
  if (initialFilters.clientId) {
    params.set("clientId", initialFilters.clientId);
  }
  if (initialFilters.projectId) {
    params.set("projectId", initialFilters.projectId);
  }
  const query = params.toString();
  return query.length > 0
    ? `/api/invoices/export?${query}`
    : "/api/invoices/export";
}, [initialFilters.clientId, initialFilters.projectId, statusFilter]);
// Filter and sort invoices based on current search term, status filter, and sort settings
const visibleInvoices = useMemo(() => {
  const query = normalizeSearch(searchTerm);
  const filteredInvoices = invoices.filter((invoice) => {
    const matchesStatus =
      statusFilter === "all" || invoice.status === statusFilter;
    const matchesSearch =
```

```
    query.length === 0 ||
    normalizeSearch(invoice.clientName).includes(query) ||
    normalizeSearch(invoice.invoiceNumber).includes(query) ||
    normalizeSearch(formatInvoiceNumber(invoice.invoiceNumber)).includes(
      query,
    );
  return matchesStatus && matchesSearch;
 });
 return sortInvoices(filteredInvoices, sortBy, sortDirection);
}, [invoices, searchTerm, sortBy, sortDirection, statusFilter]);
return (
 <div className={styles.panel}>
  <div className={styles.filterBar}>
   <div className={styles.statusFilters} aria-label="Invoice status">
    {statusFilters.map((filter) => (
     <button
      className={`${styles.filterButton} ${
       statusFilter === filter.value ? styles.activeFilter : ""
      }`}
      key={filter.value}
      onClick={() => setStatusFilter(filter.value)}
      type="button"
     >
      {filter.label}
     </button>
    ))}
```

```
</div>
<label className={styles.searchField}>
 <span>Search</span>
 <input
  onChange={(event) => setSearchTerm(event.target.value)}
  placeholder="Client or invoice number"
  type="search"
  value={searchTerm}
 />
</label>
<div className={styles.sortControls}>
 <label className={styles.selectField}>
  <span>Sort by</span>
  <select
   onChange={(event) => setSortBy(event.target.value)}
   value={sortBy}
  >
   <option value="dueDate">Due date</option>
   <option value="total">Total amount</option>
  </select>
 </label>
 <button
  className={styles.secondaryButton}
  onClick={() =>
   setSortDirection((current) =>
    current === "asc" ? "desc" : "asc",
```

```
      )
     }
     type="button"
    >
     {sortDirection === "asc" ? "Ascending" : "Descending"}
    </button>
    <a className={styles.exportButton} href={exportHref}>
     Export CSV
    </a>
   </div>
  </div>
  {!hasInvoices ? (
   <div className={styles.emptyState}>
    <p className={styles.emptyText}>
     No invoices yet. Create your first invoice to get started.
    </p>
    <Link className={styles.primaryButton} href="/invoices/new">
     Create Invoice
    </Link>
   </div>
  ) : visibleInvoices.length === 0 ? (
   <div className={styles.emptyState}>
    <p className={styles.emptyText}>
     No invoices match the current filter.
    </p>
    {hasServerFilters ? (
```

```
    <Link className={styles.secondaryButton} href="/invoices">
      Clear Filter
    </Link>
   ) : (
    <button
      className={styles.secondaryButton}
      onClick={clearClientFilters}
      type="button"
    >
      Clear Filter
    </button>
   )}
  </div>
 ) : (
  <div className={styles.tableWrap}>
   <table className={styles.table}>
    <thead>
     <tr>
      <th scope="col">Invoice</th>
      <th scope="col">Client</th>
      <th scope="col">Project</th>
      <th scope="col">Issue date</th>
      <th scope="col">Due date</th>
      <th scope="col">Status</th>
      <th scope="col">Total</th>
      <th scope="col">Actions</th>
```

```
 </tr>
</thead>
<tbody>
 {visibleInvoices.map((invoice) => (
  <tr key={invoice.id}>
   <td data-label="Invoice">
    <strong className={styles.invoiceNumber}>
     {formatInvoiceNumber(invoice.invoiceNumber)}
    </strong>
   </td>
   <td data-label="Client">{invoice.clientName}</td>
   <td data-label="Project">{invoice.projectName}</td>
   <td data-label="Issue date">
    {formatDate(invoice.issueDate)}
   </td>
   <td data-label="Due date">{formatDate(invoice.dueDate)}</td>
   <td data-label="Status">
    <StatusBadge status={invoice.status} />
   </td>
   <td data-label="Total">
    {formatCurrency(invoice.totalCents, invoice.currency)}
   </td>
   <td data-label="Actions">
    <InvoiceActions invoice={invoice} />
   </td>
  </tr>
```

```
          ))}
        </tbody>
      </table>
    </div>
  ))}
 </div>
 );
}
```

ProjectInlineForm.jsx

Found in folder: **/src/app/projects**

This code provides a form to create a new project inline, handling input validation and submission. It toggles between showing an “Add Project” button and the form itself when clicked. The form validates the hourly rate, submits data via a server action, and refreshes the page on success.

```
"use client";

import { useRouter } from "next/navigation";

import { useState, useTransition } from "react";

import { createProjectAction } from "@/actions/projectActions";

import styles from "./ProjectsPage.module.css";

// Default empty state for the form fields

const emptyFormData = {
 clientId: "",
 name: "",
 description: "",
 hourlyRate: "",
};
```

```
// Converts a formatted hourly rate string (like "75.50") to cents (7550) for storage, or returns NaN if invalid
function parseHourlyRateCents(value) {
  const text = String(value || "").trim();
  if (text === "") {
    return null; // Allow empty hourly rate
  }
  const normalizedText = text.replace(/[$£€,\s]/g, ""); // Remove currency symbols and formatting
  if (!/^\d+(\.\d{1,2})?$/.test(normalizedText)) {
    return Number.NaN; // Invalid format
  }
  return Math.round(Number(normalizedText) * 100); // Convert to cents
}
export default function ProjectInlineForm({ clients }) {
  const router = useRouter();
  const [isOpen, setIsOpen] = useState(false); // Controls whether the form is visible
  const [formData, setFormData] = useState(emptyFormData);
  const [error, setError] = useState(null);
  const [isPending, startTransition] = useTransition();
  const hasClients = clients.length > 0; // Determines if a client can be selected
  function openForm() {
    // Pre-populate clientId with the first client if available
    setFormData({
      ...emptyFormData,
      clientId: clients[0]?.id ? String(clients[0].id) : "",
    });
```

```
  setError(null);
  setIsOpen(true);
}
function closeForm() {
  setFormData(emptyFormData);
  setError(null);
  setIsOpen(false);
}
function updateField(event) {
  const { name, value } = event.target;
  setFormData((current) => ({
    ...current,
    [name]: value,
  }));
}
function submitForm(event) {
  event.preventDefault();
  setError(null);
  const hourlyRateCents = parseHourlyRateCents(formData.hourlyRate);
  if (Number.isNaN(hourlyRateCents)) {
    setError("Hourly rate must be a valid amount, such as 75 or 75.50.");
    return;
  }
  // Use transition to keep UI responsive during submission
  startTransition(async () => {
    const result = await createProjectAction({
```

```
      clientId: formData.clientId,
      name: formData.name,
      description: formData.description,
      hourlyRateCents,
    });
    if (result.error) {
      setError(result.error);
      return;
    }
    // Reset form and refresh page data on success
    setIsOpen(false);
    setFormData(emptyFormData);
    router.refresh();
  });
}
// Show button to open form when closed
if (!isOpen) {
  return (
    <button className={styles.primaryButton} type="button" onClick={openForm}>
      Add Project
    </button>
  );
}
return (
  <form className={styles.inlineForm} onSubmit={submitForm}>
    <div className={styles.formGrid}>
```

```
<label className={styles.field}>
 <span>Client</span>
 <select
  name="clientId"
  required
  value={formData.clientId}
  onChange={updateField}
  disabled={!hasClients}
 >
  {hasClients ? null : (
   <option value="">Add a client first</option>
  )}
  {clients.map((client) => (
   <option key={client.id} value={client.id}>
    {client.name}
   </option>
  ))}
 </select>
</label>
<label className={styles.field}>
 <span>Project name</span>
 <input
  name="name"
  required
  value={formData.name}
  onChange={updateField}
```

```
    />
  </label>
  <label className={`${styles.field} ${styles.fullWidth}`}>
    <span>Description</span>
    <textarea
      name="description"
      rows="2"
      value={formData.description}
      onChange={updateField}
    />
  </label>
  <label className={styles.field}>
    <span>Hourly rate</span>
    <input
      name="hourlyRate"
      inputMode="decimal"
      placeholder="75.00"
      value={formData.hourlyRate}
      onChange={updateField}
    />
  </label>
</div>
{error ? <p className={styles.error}>{error}</p> : null}
<div className={styles.formActions}>
  <button
    className={styles.primaryButton}
```

```
        type="submit"
        disabled={isPending || !hasClients}
      >
        {isPending ? "Saving..." : "Create"}
      </button>
      <button
        className={styles.secondaryButton}
        type="button"
        onClick={closeForm}
        disabled={isPending}
      >
        Cancel
      </button>
    </div>
  </form>
 );
}
```

Header.jsx

Found in folder: **/src/components**

This code renders a header component that displays a page title based on the current URL path, and includes a link to create new invoices. It uses a helper function to determine the title by matching specific route patterns. The header shows a branding subtitle and conditionally updates the main heading depending on which page the user is viewing.

```
"use client";

import Link from "next/link";

import { usePathname } from "next/navigation";

import styles from "./Header.module.css";
```

```
// Determines the appropriate page title based on the current route
function getPageTitle(pathname) {
 if (pathname === "/") {
  return "Dashboard";
 }
 if (pathname === "/clients/new") {
  return "New Client";
 }
 if (pathname.startsWith("/clients/") && pathname.endsWith("/edit")) {
  return "Edit Client";
 }
 if (pathname.startsWith("/clients/")) {
  return "Client Details";
 }
 if (pathname === "/clients") {
  return "Clients";
 }
 if (pathname === "/projects/new") {
  return "New Project";
 }
 if (pathname.startsWith("/projects/") && pathname.endsWith("/edit")) {
  return "Edit Project";
 }
 if (pathname.startsWith("/projects/")) {
  return "Project Details";
 }
```

```
  if (pathname === "/projects") {
    return "Projects";
  }
  if (pathname === "/invoices/new") {
    return "New Invoice";
  }
  if (pathname.startsWith("/invoices/") && pathname.endsWith("/edit")) {
    return "Edit Invoice";
  }
  if (pathname.startsWith("/invoices/")) {
    return "Invoice Details";
  }
  if (pathname === "/invoices") {
    return "Invoices";
  }
  // Fallback title for any unmatched routes
  return "Invoice Tracker";
}
export default function Header() {
  const pathname = usePathname();
  return (
    <header className={styles.header}>
      <div>
        <p className={styles.eyebrow}>Freelancer Bookkeeping</p>
        <h1 className={styles.title}>{getPageTitle(pathname)}</h1>
      </div>
```

```
    {/* Always shows a link to create a new invoice */}
    <Link href="/invoices/new" className={styles.action}>
      New Invoice
    </Link>
  </header>
 );
}
```

Sidebar.jsx

Found in folder: **/src/components**

This code renders a sidebar navigation component for a web application. It defines a list of navigation links and highlights the active route based on the current URL path. The sidebar includes a branded header and accessible navigation menu.

```
"use client";

import Link from "next/link";

import { usePathname } from "next/navigation";

import styles from "./Sidebar.module.css";

// Navigation items for the sidebar

const navigationItems = [

 { href: "/", label: "Dashboard" },

 { href: "/clients", label: "Clients" },

 { href: "/projects", label: "Projects" },

 { href: "/invoices", label: "Invoices" },

];

// Determines if a navigation link corresponds to the current route

function isActiveRoute(pathname, href) {

 if (href === "/") {

  return pathname === "/";
```

```
  }
  // Active for exact match or nested routes under the href
  return pathname === href || pathname.startsWith(`${href}/`);
}
export default function Sidebar() {
  const pathname = usePathname();
  return (
    <aside className={styles.sidebar}>
      {/* Brand/logo link that navigates to the homepage */}
      <Link href="/" className={styles.brand} aria-label="Invoice Tracker home">
        <span className={styles.brandMark}>IT</span>
        <span className={styles.brandText}>Invoice Tracker</span>
      </Link>
      <nav className={styles.nav} aria-label="Primary navigation">
        {navigationItems.map((item) => {
          const isActive = isActiveRoute(pathname, item.href);
          return (
            <Link
              key={item.href}
              href={item.href}
              className={`${styles.navLink} ${isActive ? styles.active : ""}`}
              aria-current={isActive ? "page" : undefined}
            >
              {item.label}
            </Link>
          );
```

```
      })}
    </nav>
  </aside>
 );
}
```

StatusBadge.jsx

Found in folder: **/src/components**

This code maps status strings to color tones and displays them as styled badges with formatted labels. It normalizes input statuses to lowercase keys for consistent lookups and applies CSS classes for styling. The badge text is formatted by replacing hyphens or underscores with spaces and capitalizing each word.

```
import styles from "./StatusBadge.module.css";

// Maps status values to color tone names used in CSS classes
const statusToneMap = {
 draft: "grey",
 sent: "blue",
 paid: "green",
 overdue: "red",
 void: "darkGrey",
 active: "green",
 paused: "yellow",
 completed: "blue",
 archived: "grey",
};
// Formats status strings for display by replacing separators and capitalizing words
function formatStatusLabel(status) {
 return String(status)
```

```
    .replace(/[-_]/g, " ")
    .replace(/\b\w/g, (letter) => letter.toUpperCase());
}
export default function StatusBadge({ status }) {
  // Normalize status to lowercase for consistent mapping, default to empty string
  const normalizedStatus = String(status || "").toLowerCase();
  // Get tone from map, default to "grey" if status not found
  const tone = statusToneMap[normalizedStatus] || "grey";
  return (
    <span className={`${styles.badge} ${styles[tone]}`}>
      {formatStatusLabel(normalizedStatus)}
    </span>
  );
}
```

Dashboard.module.css

Found in folder: **/src/app**

This CSS file defines styles for a dashboard layout, including a summary grid, cards, panels, and a responsive list for overdue items. It uses flexbox and grid for layout, with specific typography, colors, and spacing for elements like headers, titles, and data values. The code includes media queries to adjust the layout for smaller screens, changing grid columns and font sizes.

```
.page {
  display: flex;
  flex-direction: column;
  gap: 24px;
}
.header,
```

```
.panelHeader {
  display: flex;
  align-items: flex-start;
  justify-content: space-between;
  gap: 24px;
}
.eyebrow {
  margin: 0 0 8px;
  color: #64748b;
  font-size: 0.78rem;
  font-weight: 700;
  text-transform: uppercase;
}
.title {
  margin: 0;
  color: #172033;
  font-size: 2rem;
  line-height: 1.15;
}
.sectionTitle {
  margin: 0;
  color: #172033;
  font-size: 1.25rem;
  line-height: 1.2;
}
.summaryGrid {
```

```
  display: grid;
  grid-template-columns: repeat(4, minmax(0, 1fr)); /* Four equal columns by default */
  gap: 16px;
}
.summaryCard,
.panel {
  border: 1px solid #d9e1ec;
  border-radius: 8px;
  background: #ffffff;
}
.summaryCard {
  min-width: 0; /* Prevents flex/grid items from overflowing */
  padding: 18px;
}
.cardLabel {
  margin: 0 0 12px;
  color: #64748b;
  font-size: 0.82rem;
  font-weight: 700;
}
.cardValue {
  margin: 0;
  overflow-wrap: anywhere; /* Allows text to break anywhere to prevent overflow */
  color: #172033;
  font-size: 1.55rem;
  font-weight: 800;
```

```
  line-height: 1.15;
}
.dangerValue {
  color: #b91c1c; /* Red color for negative or critical values */
}
.panel {
  overflow: hidden; /* Contains child elements within rounded borders */
}
.panelHeader {
  border-bottom: 1px solid #e2e8f0;
  background: #f8fafc;
  padding: 18px;
}
.chartFrame {
  height: 320px;
  padding: 18px 18px 12px;
}
.overdueList {
  display: flex;
  flex-direction: column;
}
.overdueRow {
  display: grid;
  grid-template-columns: minmax(0, 1fr) auto; /* Client info takes available space, meta aligns to end */
  gap: 16px;
```

```
  align-items: center;
  border-bottom: 1px solid #e2e8f0;
  color: inherit;
  padding: 16px 18px;
  text-decoration: none;
}
.overdueRow:last-child {
  border-bottom: 0; /* Removes border from the last row */
}
.overdueRow:hover {
  background: #f8fafc; /* Highlight row on hover */
}
.invoiceIdentity {
  display: flex;
  min-width: 0;
  flex-direction: column;
  gap: 4px;
}
.clientName {
  overflow: hidden;
  color: #172033;
  font-weight: 800;
  text-overflow: ellipsis;
  white-space: nowrap; /* Ensures single line with ellipsis for overflow */
}
.invoiceNumber {
```

```
  color: #64748b;
  font-size: 0.88rem;
  font-weight: 700;
}
.invoiceMeta {
  display: flex;
  align-items: center;
  gap: 18px;
  color: #64748b;
  font-size: 0.92rem;
  white-space: nowrap;
}
.invoiceMeta strong {
  color: #b91c1c; /* Highlights important numbers, like overdue amounts */
}
.emptyState {
  padding: 32px;
}
.emptyTitle {
  margin: 0 0 8px;
  color: #172033;
  font-size: 1.25rem;
}
.emptyText {
  margin: 0;
  color: #64748b;
```

```
}
@media (max-width: 1100px) {
  .summaryGrid {
    grid-template-columns: repeat(2, minmax(0, 1fr)); /* Two columns on medium screens */
  }
}
@media (max-width: 700px) {
  .title {
    font-size: 1.65rem; /* Smaller title on mobile */
  }
  .summaryGrid {
    grid-template-columns: 1fr; /* Single column on small screens */
  }
  .overdueRow {
    grid-template-columns: 1fr; /* Stack invoice identity and meta vertically */
  }
  .invoiceMeta {
    justify-content: space-between; /* Spread out meta items on mobile */
  }
}
```

Error.module.css

Found in folder: **/src/app**

This CSS defines styles for an error state component, including a container, message text, and a button. The container has a bordered card layout with padding and spacing, while the button includes hover effects for interactivity. These styles ensure visual consistency and user feedback within the interface.

```
.errorState {
```

```
  display: flex;
  max-width: 640px;
  flex-direction: column;
  align-items: flex-start;
  gap: 16px;
  border: 1px solid #d9e1ec;
  border-radius: 8px;
  background: #ffffff;
  padding: 32px;
}
.message {
  margin: 0;
  color: #334155;
  font-size: 1rem;
  line-height: 1.5;
}
.button {
  display: inline-flex;
  min-height: 40px;
  align-items: center;
  justify-content: center;
  border: 1px solid #cbd5e1;
  border-radius: 8px;
  background: #ffffff;
  color: #334155;
  cursor: pointer;
```

```
  font-weight: 700;
  padding: 0 14px;
}
.button:hover {
  background: #f8fafc; /* Light background on hover for visual feedback */
}
```

globals.css

Found in folder: **/src/app**

This CSS file sets global styles for the project, ensuring consistent box-sizing and establishing a full-height layout. It defines a base color scheme and font family for the body, and resets margins for a clean start. Additionally, it ensures form elements inherit the font settings from the body for visual consistency.

```
/* Apply border-box sizing to all elements for predictable layout calculations */
* {
  box-sizing: border-box;
}
/* Ensure the html element always spans at least the full viewport height */
html {
  min-height: 100%;
}
/* Set base styles for the body: full height, no margin, background/foreground colors, and default font stack */
body {
  min-height: 100%;
  margin: 0;
  background: #f1f5f9;
  color: #172033;
```

```
  font-family:
    Arial,
    Helvetica,
    sans-serif;
}
/* Make form elements inherit the font family and size from the body for consistency */
button,
input,
select,
textarea {
  font: inherit;
}
```

layout.module.css

Found in folder: **/src/app**

This CSS defines a responsive layout with a sidebar and main content area, adjusting for mobile screens and print media. It uses CSS Grid for the overall structure and Flexbox for inner content alignment. The print styles hide non-essential elements and optimize the page for printing.

```
.shell {
  display: grid;
  min-height: 100vh; /* Full viewport height */
  grid-template-columns: 260px minmax(0, 1fr); /* Sidebar and flexible main column */
  background: #f1f5f9;
}
.content {
  display: flex;
  min-width: 0; /* Prevents flex overflow issues */
```

```
  flex-direction: column;
}
.main {
  flex: 1; /* Takes remaining vertical space */
  padding: 32px;
}
@media (max-width: 760px) {
  .shell {
    grid-template-columns: 1fr; /* Collapses to single column on mobile */
  }
  .main {
    padding: 20px; /* Reduced padding for smaller screens */
  }
}
@media print {
  .shell {
    display: block;
    min-height: auto;
    background: #ffffff; /* White background for printing */
  }
  .shell > aside,
  .content > header {
    display: none; /* Hide sidebar and header when printing */
  }
  .content {
    display: block;
```

```
  }
  .main {
    padding: 0; /* Remove padding for print layout */
  }
}
```

ClientsPage.module.css

Found in folder: **/src/app/clients**

This CSS file defines styles for a responsive web interface, including layout containers, a data table, form elements, and interactive buttons. It uses media queries to adapt the layout for mobile and tablet screens, converting the table into a stacked list view. The styles also manage visual states like hover effects, disabled buttons, and form validation errors.

```
.page {
  display: flex;
  flex-direction: column;
  gap: 24px;
}
.header {
  display: flex;
  align-items: flex-start;
  justify-content: space-between;
  gap: 24px;
}
.eyebrow {
  margin: 0 0 8px;
  color: #64748b;
  font-size: 0.78rem;
  font-weight: 700;
```

```
  text-transform: uppercase;
}
.title {
  margin: 0;
  color: #172033;
  font-size: 2rem;
  line-height: 1.15;
}
.panel {
  overflow: hidden;
  border: 1px solid #d9e1ec;
  border-radius: 8px;
  background: #ffffff;
}
.tableWrap {
  overflow-x: auto; /* Allows horizontal scrolling for wide tables */
}
.table {
  width: 100%;
  border-collapse: collapse;
  font-size: 0.95rem;
}
.table th,
.table td {
  border-bottom: 1px solid #e2e8f0;
  padding: 16px;
```

```
  text-align: left;
  vertical-align: top;
}
.table th {
  background: #f8fafc;
  color: #526071;
  font-size: 0.78rem;
  font-weight: 700;
  text-transform: uppercase;
  white-space: nowrap;
}
.table tr:last-child td {
  border-bottom: 0;
}
.clientName {
  color: #172033;
}
.projectCount {
  display: inline-flex;
  align-items: center;
  gap: 8px;
  white-space: nowrap;
}
.primaryButton,
.secondaryButton {
  display: inline-flex;
```

```
  min-height: 40px;
  align-items: center;
  justify-content: center;
  border: 1px solid transparent;
  border-radius: 8px;
  cursor: pointer;
  font-weight: 700;
  padding: 0 14px;
  text-decoration: none;
  white-space: nowrap;
}
.primaryButton {
  background: #1d4ed8;
  color: #ffffff;
}
.primaryButton:hover {
  background: #1e40af;
}
.secondaryButton {
  border-color: #cbd5e1;
  background: #ffffff;
  color: #334155;
}
.secondaryButton:hover {
  background: #f8fafc;
}
```

```
.primaryButton:disabled,
.secondaryButton:disabled {
  cursor: not-allowed;
  opacity: 0.65;
}
.inlineForm {
  min-width: min(640px, calc(100vw - 64px));
  border: 1px solid #d9e1ec;
  border-radius: 8px;
  background: #f8fafc;
  padding: 16px;
}
.editForm {
  min-width: min(520px, calc(100vw - 96px));
}
.formGrid {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 14px;
}
.field {
  display: flex;
  min-width: 0;
  flex-direction: column;
  gap: 6px;
  color: #526071;
```

```
  font-size: 0.82rem;
  font-weight: 700;
}
.field input,
.field textarea {
  width: 100%;
  border: 1px solid #cbd5e1;
  border-radius: 8px;
  background: #ffffff;
  color: #172033;
  padding: 10px 12px;
}
.field textarea {
  resize: vertical;
}
.field input:focus,
.field textarea:focus {
  border-color: #1d4ed8;
  outline: 2px solid #bfdbfe;
  outline-offset: 1px;
}
.fullWidth {
  grid-column: 1 / -1; /* Spans across all columns in the form grid */
}
.formActions {
  display: flex;
```

```
  flex-wrap: wrap;
  gap: 10px;
  margin-top: 14px;
}
.error {
  margin: 12px 0 0;
  border-radius: 8px;
  background: #fee2e2;
  color: #991b1b;
  font-size: 0.9rem;
  font-weight: 700;
  padding: 10px 12px;
}
.emptyState {
  display: flex;
  flex-direction: column;
  align-items: flex-start;
  gap: 16px;
  padding: 32px;
}
.emptyText {
  margin: 0;
  color: #64748b;
}
/* Responsive styles for tablet screens */
@media (max-width: 860px) {
```

```
.header {
  flex-direction: column;
}
/* Converts the table into a block layout for mobile */
.table,
.table thead,
.table tbody,
.table th,
.table td,
.table tr {
  display: block;
}
.table thead {
  display: none;
}
.table tr {
  border-bottom: 1px solid #e2e8f0;
  padding: 14px 16px;
}
.table tr:last-child {
  border-bottom: 0;
}
.table td {
  display: grid;
  grid-template-columns: minmax(120px, 42%) minmax(0, 1fr);
  gap: 12px;
```

```
  border-bottom: 0;
  padding: 8px 0;
 }
 /* Uses data-label attribute to show column headers */
 .table td::before {
  content: attr(data-label);
  color: #64748b;
  font-size: 0.78rem;
  font-weight: 700;
  text-transform: uppercase;
 }
 .inlineForm,
 .editForm {
  min-width: 0;
  width: 100%;
 }
}
/* Responsive styles for mobile screens */
@media (max-width: 560px) {
 .title {
  font-size: 1.65rem;
 }
 .formGrid {
  grid-template-columns: 1fr; /* Single column layout */
 }
 .table td {
```

```
  grid-template-columns: 1fr;
  gap: 6px;
 }
 .primaryButton,
 .secondaryButton {
  width: 100%; /* Full-width buttons on small screens */
 }
}
```

Error.module.css

Found in folder: **/src/app/clients**

This CSS defines styles for an error state container, its message text, and a button. The error state is a bordered box with internal spacing, while the button has interactive hover effects. The styles focus on layout, color, and visual feedback.

```
.errorState {
 display: flex;
 flex-direction: column;
 align-items: flex-start;
 gap: 16px;
 border: 1px solid #d9e1ec;
 border-radius: 8px;
 background: #ffffff;
 padding: 32px;
}
.message {
 margin: 0;
 color: #334155;
 font-size: 1rem;
```

```
  line-height: 1.5;
}
.button {
  display: inline-flex;
  min-height: 40px;
  align-items: center;
  justify-content: center;
  border: 1px solid #cbd5e1;
  border-radius: 8px;
  background: #ffffff;
  color: #334155;
  cursor: pointer;
  font-weight: 700;
  padding: 0 14px;
}
.button:hover {
  background: #f8fafc; /* Light background on hover for visual feedback */
}
```

DeleteConfirmModal.module.css

Found in folder: **/src/app/invoices**

This CSS styles a modal component with a backdrop, content area, title, message, action buttons, and error display. It defines visual properties like colors, spacing, and hover states for interactive elements. The styles ensure the modal is centered with a semi-transparent overlay and includes responsive width constraints.

```
.backdrop {
  position: fixed;
  z-index: 50;
```

```
  inset: 0; /* Covers entire viewport */
  display: flex;
  align-items: center;
  justify-content: center;
  background: rgb(15 23 42 / 0.52); /* Semi-transparent overlay */
  padding: 24px;
}
.modal {
  width: min(420px, 100%); /* Responsive width, max 420px */
  border: 1px solid #d9e1ec;
  border-radius: 8px;
  background: #ffffff;
  box-shadow: 0 24px 48px rgb(15 23 42 / 0.22);
  padding: 20px;
}
.title {
  margin: 0;
  color: #172033;
  font-size: 1.2rem;
  line-height: 1.3;
}
.message {
  margin: 12px 0 0;
  color: #334155;
  line-height: 1.5;
}
```

```
.actions {
  display: flex;
  flex-wrap: wrap;
  gap: 10px;
  justify-content: flex-end; /* Aligns buttons to the right */
  margin-top: 20px;
}
.dangerButton,
.secondaryButton {
  display: inline-flex;
  min-height: 40px;
  align-items: center;
  justify-content: center;
  border: 1px solid transparent;
  border-radius: 8px;
  cursor: pointer;
  font-weight: 700;
  padding: 0 14px;
  white-space: nowrap; /* Prevents text wrapping */
}
.dangerButton {
  background: #b91c1c;
  color: #ffffff;
}
.dangerButton:hover {
  background: #991b1b;
```

```
}
.secondaryButton {
  border-color: #cbd5e1;
  background: #ffffff;
  color: #334155;
}
.secondaryButton:hover {
  background: #f8fafc;
}
.dangerButton:disabled,
.secondaryButton:disabled {
  cursor: not-allowed;
  opacity: 0.65;
}
.error {
  margin: 12px 0 0;
  border-radius: 8px;
  background: #fee2e2;
  color: #991b1b;
  font-size: 0.9rem;
  font-weight: 700;
  padding: 10px 12px;
}
```

Error.module.css

Found in folder: **/src/app/invoices**

This CSS defines styles for an error state container, its message text, and a button. The `.errorState` class creates a bordered box with internal spacing, while `.button` styles an interactive element with hover effects. The `.message` class sets typography for text within the error container.

```
.errorState {
  display: flex;
  flex-direction: column;
  align-items: flex-start;
  gap: 16px;
  border: 1px solid #d9e1ec;
  border-radius: 8px;
  background: #ffffff;
  padding: 32px;
}
.message {
  margin: 0;
  color: #334155;
  font-size: 1rem;
  line-height: 1.5;
}
.button {
  display: inline-flex;
  min-height: 40px;
  align-items: center;
  justify-content: center;
  border: 1px solid #cbd5e1;
  border-radius: 8px;
```

```
  background: #ffffff;
  color: #334155;
  cursor: pointer;
  font-weight: 700;
  padding: 0 14px;
}
.button:hover {
  background: #f8fafc; /* Light background on hover for visual feedback */
}
```

InvoiceActions.module.css

Found in folder: **/src/app/invoices**

This CSS file defines styles for action buttons and error messages. It sets up a flex container for button groups and creates four button types with distinct colors and hover effects. It also includes a disabled state for all buttons and a styled error message box.

```
.actions {
  display: flex;
  flex-wrap: wrap;
  gap: 10px; /* Spacing between buttons in the container */
}
.primaryButton,
.secondaryButton,
.tertiaryButton,
.dangerButton {
  display: inline-flex;
  min-height: 40px;
  align-items: center;
  justify-content: center;
```

```
  border: 1px solid transparent;
  border-radius: 8px;
  cursor: pointer;
  font-weight: 700;
  padding: 0 14px;
  text-decoration: none;
  white-space: nowrap;
}
.primaryButton {
  background: #1d4ed8;
  color: #ffffff;
}
.primaryButton:hover {
  background: #1e40af; /* Darker blue on hover */
}
.secondaryButton {
  border-color: #cbd5e1;
  background: #ffffff;
  color: #334155;
}
.secondaryButton:hover {
  background: #eef2f7; /* Light gray background on hover */
}
.tertiaryButton {
  border-color: #e2e8f0;
  background: #ffffff;
```

```
  color: #64748b;
}
.tertiaryButton:hover {
  background: #f8fafc;
  color: #334155; /* Darker text color on hover */
}
.dangerButton {
  background: #b91c1c;
  color: #ffffff;
}
.dangerButton:hover {
  background: #991b1b; /* Darker red on hover */
}
.primaryButton:disabled,
.secondaryButton:disabled,
.tertiaryButton:disabled,
.dangerButton:disabled {
  cursor: not-allowed;
  opacity: 0.65; /* Visual indication for disabled state */
}
.error {
  margin: 10px 0 0;
  border-radius: 8px;
  background: #fee2e2; /* Light red background */
  color: #991b1b;
  font-size: 0.9rem;
```

```
  font-weight: 700;
  padding: 10px 12px;
}
```

InvoiceForm.module.css

Found in folder: **/src/app/invoices**

This CSS defines the visual layout and styling for a form interface, including sections, input fields, line items, and buttons. It styles form elements with specific colors, borders, and spacing, and includes responsive design rules for different screen sizes. The code ensures the form is accessible with focus states and disabled states for interactive elements.

```
.form {
  display: flex;
  max-width: 1120px;
  flex-direction: column;
  gap: 20px;
}
.panel {
  overflow: hidden;
  border: 1px solid #d9e1ec;
  border-radius: 8px;
  background: #ffffff;
}
.sectionHeader {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 16px;
  border-bottom: 1px solid #e2e8f0;
```

```
  background: #f8fafc;
  padding: 18px 20px;
}
.eyebrow {
  margin: 0 0 6px;
  color: #64748b;
  font-size: 0.75rem;
  font-weight: 700;
  text-transform: uppercase;
}
.sectionTitle {
  margin: 0;
  color: #172033;
  font-size: 1.2rem;
  line-height: 1.2;
}
.formGrid {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr)); /* Two-column grid */
  gap: 18px;
  padding: 20px;
}
.field,
.lineField {
  display: flex;
  min-width: 0;
```

```
  flex-direction: column;
  gap: 7px;
  color: #526071;
  font-size: 0.82rem;
  font-weight: 700;
}
/* Styling for input and select elements */
.field input,
.field select,
.lineField input {
  width: 100%;
  min-height: 42px;
  border: 1px solid #cbd5e1;
  border-radius: 8px;
  background: #ffffff;
  color: #172033;
  padding: 8px 12px;
}
.field input:focus,
.field select:focus,
.lineField input:focus {
  border-color: #1d4ed8;
  outline: 2px solid #bfdbfe;
  outline-offset: 1px;
}
.field input:disabled,
```

```
.field select:disabled,
.lineField input:disabled {
  background: #f1f5f9;
  color: #64748b;
  cursor: not-allowed;
}
.lineItems {
  display: flex;
  flex-direction: column;
}
/* Grid layout for line item headers and rows */
.lineItemHeader,
.lineItemRow {
  display: grid;
  grid-template-columns:
    minmax(220px, 1.6fr)
    minmax(110px, 0.55fr)
    minmax(140px, 0.65fr)
    minmax(130px, 0.55fr)
    minmax(96px, auto);
  gap: 12px;
  align-items: end;
}
.lineItemHeader {
  border-bottom: 1px solid #e2e8f0;
  color: #64748b;
```

```
  font-size: 0.76rem;
  font-weight: 700;
  padding: 12px 20px;
  text-transform: uppercase;
}
.lineItemRow {
  border-bottom: 1px solid #e2e8f0;
  padding: 16px 20px;
}
.lineItemRow:last-child {
  border-bottom: 0;
}
.lineField span,
.lineTotal span {
  display: none; /* Hidden labels for desktop view */
}
.lineTotal {
  display: flex;
  min-height: 42px;
  align-items: center;
  color: #172033;
  font-size: 0.95rem;
}
.totalRow {
  display: flex;
  align-items: center;
```

```
  justify-content: flex-end;
  gap: 24px;
  border-top: 1px solid #d9e1ec;
  background: #f8fafc;
  color: #172033;
  font-size: 1.05rem;
  padding: 18px 20px;
}
.totalRow span {
  color: #526071;
  font-size: 0.82rem;
  font-weight: 700;
  text-transform: uppercase;
}
/* Base button styles */
.primaryButton,
.secondaryButton,
.dangerButton {
  display: inline-flex;
  min-height: 40px;
  align-items: center;
  justify-content: center;
  border: 1px solid #cbd5e1;
  border-radius: 8px;
  cursor: pointer;
  font-weight: 700;
```

```
  padding: 0 14px;
  text-decoration: none;
  white-space: nowrap;
}
.primaryButton {
  border-color: #1d4ed8;
  background: #1d4ed8;
  color: #ffffff;
}
.primaryButton:hover {
  background: #1e40af;
}
.secondaryButton {
  background: #ffffff;
  color: #334155;
}
.secondaryButton:hover {
  background: #eef2f7;
}
.dangerButton {
  border-color: #fecaca;
  background: #ffffff;
  color: #991b1b;
}
.dangerButton:hover {
  background: #fef2f2;
```

```
}
.primaryButton:disabled,
.secondaryButton:disabled,
.dangerButton:disabled {
  cursor: not-allowed;
  opacity: 0.65;
}
.error {
  margin: 0;
  border: 1px solid #fecaca;
  border-radius: 8px;
  background: #fef2f2;
  color: #991b1b;
  font-weight: 700;
  padding: 12px 14px;
}
.formActions {
  display: flex;
  flex-wrap: wrap;
  gap: 10px;
}
/* Responsive design for tablets and smaller screens */
@media (max-width: 980px) {
  .lineItemHeader {
    display: none; /* Hide header on smaller screens */
  }
```

```
  .lineItemRow {
    grid-template-columns: repeat(2, minmax(0, 1fr)); /* Switch to two-column layout */
    align-items: stretch;
  }
  .lineField span,
  .lineTotal span {
    display: inline; /* Show labels for mobile view */
  }
  .lineTotal {
    flex-direction: column;
    align-items: flex-start;
    gap: 7px;
    color: #526071;
    font-size: 0.82rem;
    font-weight: 700;
  }
  .lineTotal strong {
    color: #172033;
    font-size: 0.95rem;
  }
}
/* Responsive design for mobile screens */
@media (max-width: 680px) {
  .formGrid,
  .lineItemRow {
    grid-template-columns: 1fr; /* Single column layout */
```

```
 }
 .sectionHeader,
 .formActions {
  align-items: stretch;
  flex-direction: column;
 }
 .totalRow {
  justify-content: space-between;
 }
}
```

InvoicesPage.module.css

Found in folder: **/src/app/invoices**

This CSS styles a responsive dashboard interface with a filter bar, data table, and action buttons. It defines layout, colors, and interactive states for components like buttons, search fields, and a table that adapts for mobile. Media queries adjust the grid and table into stacked, single-column layouts on smaller screens.

```
.page {
 display: flex;
 flex-direction: column;
 gap: 24px;
}
.header {
 display: flex;
 align-items: flex-start;
 justify-content: space-between;
 gap: 24px;
}
```

```
.eyebrow {
  margin: 0 0 8px;
  color: #64748b;
  font-size: 0.78rem;
  font-weight: 700;
  text-transform: uppercase;
}
.title {
  margin: 0;
  color: #172033;
  font-size: 2rem;
  line-height: 1.15;
}
.panel {
  overflow: hidden;
  border: 1px solid #d9e1ec;
  border-radius: 8px;
  background: #ffffff;
}
.filterBar {
  display: grid;
  grid-template-columns: minmax(0, 1fr) minmax(220px, 300px) auto;
  gap: 16px;
  align-items: end;
  border-bottom: 1px solid #e2e8f0;
  background: #f8fafc;
```

```
  padding: 16px;
}
.statusFilters {
  display: flex;
  flex-wrap: wrap;
  gap: 8px;
}
/* Base styles for all button variants */
.filterButton,
.primaryButton,
.secondaryButton,
.exportButton,
.dangerButton {
  display: inline-flex;
  min-height: 40px;
  align-items: center;
  justify-content: center;
  border: 1px solid #cbd5e1;
  border-radius: 8px;
  background: #ffffff;
  color: #334155;
  cursor: pointer;
  font-weight: 700;
  padding: 0 14px;
  text-decoration: none;
  white-space: nowrap;
```

```
}
.primaryButton {
  border-color: #1d4ed8;
  background: #1d4ed8;
  color: #ffffff;
}
.primaryButton:hover {
  background: #1e40af;
}
.filterButton:hover,
.secondaryButton:hover,
.exportButton:hover {
  background: #eef2f7;
}
/* Style for an active filter button */
.activeFilter {
  border-color: #1d4ed8;
  background: #1d4ed8;
  color: #ffffff;
}
.activeFilter:hover {
  background: #1e40af;
}
.dangerButton {
  border-color: #fecaca;
  color: #991b1b;
```

```
}
.dangerButton:hover {
  background: #fef2f2;
}
.secondaryButton:disabled,
.dangerButton:disabled {
  cursor: not-allowed;
  opacity: 0.65;
}
.searchField,
.selectField {
  display: flex;
  min-width: 0;
  flex-direction: column;
  gap: 6px;
  color: #526071;
  font-size: 0.82rem;
  font-weight: 700;
}
.searchField input,
.selectField select {
  width: 100%;
  min-height: 40px;
  border: 1px solid #cbd5e1;
  border-radius: 8px;
  background: #ffffff;
```

```
  color: #172033;
  padding: 8px 12px;
}
.searchField input:focus,
.selectField select:focus {
  border-color: #1d4ed8;
  outline: 2px solid #bfdbfe;
  outline-offset: 1px;
}
.sortControls {
  display: flex;
  gap: 10px;
  align-items: end;
}
.tableWrap {
  overflow-x: auto; /* Allows horizontal scrolling on small screens */
}
.table {
  width: 100%;
  border-collapse: collapse;
  font-size: 0.95rem;
}
.table th,
.table td {
  border-bottom: 1px solid #e2e8f0;
  padding: 16px;
```

```
  text-align: left;
  vertical-align: top;
}
.table th {
  color: #526071;
  font-size: 0.78rem;
  font-weight: 700;
  text-transform: uppercase;
  white-space: nowrap;
}
.table tr:last-child td {
  border-bottom: 0;
}
.invoiceNumber {
  color: #172033;
  white-space: nowrap;
}
.rowActions {
  display: flex;
  flex-wrap: wrap;
  gap: 8px;
}
.emptyState {
  display: flex;
  flex-direction: column;
  align-items: flex-start;
```

```
  gap: 16px;
  padding: 32px;
}
.emptyText {
  margin: 0;
  color: #64748b;
}
/* Responsive layout adjustments */
@media (max-width: 980px) {
  .filterBar {
    grid-template-columns: 1fr; /* Stack filter bar sections */
  }
  .sortControls {
    flex-wrap: wrap;
  }
}
@media (max-width: 860px) {
  .header {
    flex-direction: column;
  }
  /* Convert table to a block layout for mobile */
  .table,
  .table thead,
  .table tbody,
  .table th,
  .table td,
```

```
.table tr {
  display: block;
}
.table thead {
  display: none; /* Hide table headers */
}
.table tr {
  border-bottom: 1px solid #e2e8f0;
  padding: 14px 16px;
}
.table tr:last-child {
  border-bottom: 0;
}
.table td {
  display: grid;
  grid-template-columns: minmax(120px, 42%) minmax(0, 1fr);
  gap: 12px;
  border-bottom: 0;
  padding: 8px 0;
}
/* Use data-label attribute to show column headers */
.table td::before {
  content: attr(data-label);
  color: #64748b;
  font-size: 0.78rem;
  font-weight: 700;
```

```
    text-transform: uppercase;
  }
}
@media (max-width: 560px) {
  .title {
    font-size: 1.65rem;
  }
  /* Stack button groups vertically on very small screens */
  .statusFilters,
  .sortControls,
  .rowActions {
    align-items: stretch;
    flex-direction: column;
  }
}
```

InvoiceDetailPage.module.css

Found in folder: **/src/app/invoices/[invoiceId]**

This CSS styles an invoice document with a responsive layout, defining visual styles for elements like buttons, headers, client information, line items, and totals. It includes interactive hover states for buttons and links, and adjusts the layout for different screen sizes using media queries. For printing, it hides interactive elements and optimizes the document's appearance with black-and-white styling and proper page breaks.

```
.page {
  display: flex;
  flex-direction: column;
  gap: 18px;
}
```

```
.actions {
  display: flex;
  flex-wrap: wrap;
  justify-content: flex-end;
  gap: 10px;
}
/* Base styles for all button and link variants */
.primaryLink,
.secondaryLink,
.primaryButton,
.secondaryButton,
.dangerButton {
  display: inline-flex;
  min-height: 40px;
  align-items: center;
  justify-content: center;
  border: 1px solid #cbd5e1;
  border-radius: 8px;
  font-weight: 700;
  padding: 0 14px;
  text-decoration: none;
  white-space: nowrap;
}
/* Primary button/link styling */
.primaryButton,
.primaryLink {
```

```
  border-color: #1d4ed8;
  background: #1d4ed8;
  color: #ffffff;
}
.primaryButton:hover,
.primaryLink:hover {
  background: #1e40af;
}
/* Secondary button/link styling */
.secondaryButton,
.secondaryLink {
  background: #ffffff;
  color: #334155;
}
.secondaryButton:hover,
.secondaryLink:hover {
  background: #eef2f7;
}
/* Danger button styling */
.dangerButton {
  border-color: #fecaca;
  background: #ffffff;
  color: #991b1b;
}
.dangerButton:hover {
  background: #fef2f2;
```

```
}
/* Ensure buttons show pointer cursor */
.primaryButton,
.secondaryButton,
.dangerButton {
  cursor: pointer;
}
/* Main invoice document container */
.document {
  width: min(100%, 980px);
  margin: 0 auto;
  border: 1px solid #d9e1ec;
  border-radius: 8px;
  background: #ffffff;
  color: #172033;
  padding: 56px;
  box-shadow: 0 24px 60px rgba(15, 23, 42, 0.08);
}
/* Header section with brand and invoice metadata */
.documentHeader {
  display: grid;
  grid-template-columns: minmax(0, 1fr) auto;
  gap: 48px;
  align-items: start;
  border-bottom: 2px solid #172033;
  padding-bottom: 36px;
```

```
}
.brandLabel,
.invoiceNumber,
.clientName,
.projectName,
.amountDue strong {
  color: #111827;
}
.brandLabel {
  margin: 0;
  font-size: 2.25rem;
  font-weight: 800;
  line-height: 1.05;
}
.tagline {
  margin: 12px 0 0;
  color: #64748b;
  font-size: 1rem;
}
.invoiceMeta {
  min-width: 250px;
  text-align: right;
}
.invoiceLabel {
  margin: 0 0 6px;
  color: #64748b;
```

```
  font-size: 0.78rem;
  font-weight: 800;
  letter-spacing: 0;
  text-transform: uppercase;
}
.invoiceNumber {
  margin: 0 0 22px;
  font-size: 2rem;
  line-height: 1.1;
}
.metaList {
  display: grid;
  gap: 10px;
  margin: 0;
}
.metaList div {
  display: grid;
  grid-template-columns: 95px minmax(0, 1fr);
  gap: 16px;
  align-items: center;
}
.metaList dt {
  color: #64748b;
  font-size: 0.78rem;
  font-weight: 800;
  text-transform: uppercase;
```

```
}
.metaList dd {
  margin: 0;
  font-weight: 700;
}
/* Client and project information sections */
.parties {
  display: grid;
  grid-template-columns: minmax(0, 1fr) minmax(280px, 0.8fr);
  gap: 48px;
  padding: 44px 0;
}
.billTo,
.project,
.payment,
.notes {
  min-width: 0;
}
.billTo h2,
.project h2,
.payment h2,
.notes h2,
.sectionTitle {
  margin: 0 0 14px;
  color: #64748b;
  font-size: 0.82rem;
```

```
  font-weight: 800;
  letter-spacing: 0;
  text-transform: uppercase;
}
.clientBlock {
  color: #475569;
  line-height: 1.6;
}
.clientName,
.projectName {
  margin: 0 0 6px;
  font-size: 1.2rem;
  font-weight: 800;
  line-height: 1.3;
}
.contactLine,
.address,
.projectNote {
  margin: 0;
}
.address {
  max-width: 34rem;
  white-space: pre-line;
}
.projectNote {
  color: #475569;
```

```
  line-height: 1.6;
}
/* Line items table section */
.lineItems {
  border-top: 1px solid #e2e8f0;
  padding-top: 34px;
}
.tableWrap {
  overflow-x: auto;
}
.table {
  width: 100%;
  border-collapse: collapse;
  font-size: 0.96rem;
}
.table th,
.table td {
  border-bottom: 1px solid #e2e8f0;
  padding: 17px 0;
  text-align: left;
  vertical-align: top;
}
.table thead th {
  color: #64748b;
  font-size: 0.78rem;
  font-weight: 800;
```

```
  text-transform: uppercase;
}
/* Align numeric columns to the right */
.table th:nth-child(n + 2),
.table td:nth-child(n + 2),
.table tfoot th,
.table tfoot td {
  text-align: right;
}
.table tbody td:first-child {
  width: 52%;
  color: #172033;
  font-weight: 600;
}
.table tfoot th,
.table tfoot td {
  border-bottom: 0;
  padding: 14px 0 0;
}
.table tfoot th {
  color: #475569;
  font-weight: 700;
}
.totalRow th,
.totalRow td {
  color: #111827;
```

```
  font-size: 1.1rem;
  font-weight: 800;
}
/* Payment details section */
.payment {
  display: grid;
  grid-template-columns: minmax(0, 1fr) auto;
  gap: 32px;
  align-items: end;
  border-top: 2px solid #172033;
  margin-top: 34px;
  padding-top: 30px;
}
.paymentDueDate {
  margin: 0;
  color: #475569;
}
.amountDue {
  min-width: 260px;
  text-align: right;
}
.amountDue p {
  margin: 0 0 8px;
  color: #64748b;
  font-size: 0.82rem;
  font-weight: 800;
```

```
  text-transform: uppercase;
}
.amountDue strong {
  display: block;
  font-size: 2rem;
  line-height: 1.1;
}
/* Notes section */
.notes {
  border-top: 1px solid #e2e8f0;
  margin-top: 32px;
  padding-top: 28px;
}
.notes p {
  margin: 0;
  color: #475569;
  line-height: 1.65;
  white-space: pre-line;
}
/* Responsive adjustments for medium screens */
@media (max-width: 820px) {
  .document {
    padding: 32px;
  }
  /* Switch grid layouts to single column */
  .documentHeader,
```

```
  .parties,
  .payment {
    grid-template-columns: 1fr;
  }
  .invoiceMeta,
  .amountDue {
    min-width: 0;
    text-align: left;
  }
  .metaList div {
    grid-template-columns: 105px minmax(0, 1fr);
  }
}
/* Responsive adjustments for small screens */
@media (max-width: 560px) {
  .actions {
    flex-direction: column-reverse;
  }
  .primaryLink,
  .secondaryLink {
    width: 100%;
  }
  .document {
    padding: 24px;
  }
  .brandLabel,
```

```
  .invoiceNumber,
  .amountDue strong {
    font-size: 1.65rem;
  }
  .documentHeader {
    gap: 28px;
  }
  .parties {
    gap: 30px;
    padding: 34px 0;
  }
  /* Ensure table remains scrollable on very small screens */
  .table {
    min-width: 620px;
  }
}
/* Print-specific styles */
@media print {
  @page {
    margin: 15mm;
  }
  .actions {
    display: none;
  }
  .page {
    display: block;
```

```
}
.document {
  width: 100%;
  max-width: none;
  border: 0;
  border-radius: 0;
  background: #ffffff;
  color: #000000;
  box-shadow: none;
  padding: 0;
}
/* Force all elements to print in black and white */
.document *,
.document *::before,
.document *::after {
  background: #ffffff;
  color: #000000;
  box-shadow: none;
}
.documentHeader,
.lineItems,
.payment,
.notes {
  border-color: #000000;
}
.tableWrap {
```

```
  overflow: visible;
 }
 .table {
  border: 1px solid #000000;
  page-break-inside: auto;
 }
 .table th,
 .table td {
  border: 1px solid #000000;
  padding: 10px;
 }
 /* Prevent page breaks inside key elements */
 .table tr,
 .table tbody,
 .table tfoot {
  break-inside: avoid;
  page-break-inside: avoid;
 }
 .payment {
  break-inside: avoid;
  page-break-inside: avoid;
 }
}
```

Error.module.css

Found in folder: **/src/app/projects**

This CSS defines styles for an error state component, including a container, a message, and a button. The container is a white bordered box with internal spacing, while the button changes background color on hover.

```
.errorState {
  display: flex;
  flex-direction: column;
  align-items: flex-start;
  gap: 16px;
  border: 1px solid #d9e1ec;
  border-radius: 8px;
  background: #ffffff;
  padding: 32px;
}
.message {
  margin: 0;
  color: #334155;
  font-size: 1rem;
  line-height: 1.5;
}
.button {
  display: inline-flex;
  min-height: 40px;
  align-items: center;
  justify-content: center;
  border: 1px solid #cbd5e1;
  border-radius: 8px;
  background: #ffffff;
```

```
  color: #334155;
  cursor: pointer;
  font-weight: 700;
  padding: 0 14px;
}
.button:hover {
  background: #f8fafc; /* Subtle hover effect for interactivity */
}
```

ProjectsPage.module.css

Found in folder: **/src/app/projects**

This CSS styles a client dashboard page with a responsive layout, including a header, client sections with tables, and form components. It defines visual styles for elements like buttons, tables, and form fields, with hover states and disabled states. The code includes responsive breakpoints that adjust the layout for tablets and mobile screens.

```
.page {
  display: flex;
  flex-direction: column;
  gap: 24px;
}
.header {
  display: flex;
  align-items: flex-start;
  justify-content: space-between;
  gap: 24px;
}
.eyebrow {
  margin: 0 0 8px;
```

```
  color: #64748b;
  font-size: 0.78rem;
  font-weight: 700;
  text-transform: uppercase;
}
.title {
  margin: 0;
  color: #172033;
  font-size: 2rem;
  line-height: 1.15;
}
.panel {
  overflow: hidden;
  border: 1px solid #d9e1ec;
  border-radius: 8px;
  background: #ffffff;
}
.clientSections {
  display: flex;
  flex-direction: column;
}
.clientSection {
  border-bottom: 1px solid #d9e1ec;
}
.clientSection:last-child {
  border-bottom: 0; /* Removes border from the last section */
```

```
}
.clientHeader {
  border-bottom: 1px solid #e2e8f0;
  background: #f8fafc;
  padding: 16px;
}
.clientName {
  margin: 0;
  color: #172033;
  font-size: 1rem;
}
.tableWrap {
  overflow-x: auto; /* Allows horizontal scrolling on small screens */
}
.table {
  width: 100%;
  border-collapse: collapse;
  font-size: 0.95rem;
}
.table th,
.table td {
  border-bottom: 1px solid #e2e8f0;
  padding: 16px;
  text-align: left;
  vertical-align: top;
}
```

```
.table th {
  color: #526071;
  font-size: 0.78rem;
  font-weight: 700;
  text-transform: uppercase;
  white-space: nowrap;
}
.table tr:last-child td {
  border-bottom: 0; /* Removes border from the last row */
}
.projectSummary {
  display: flex;
  min-width: 0;
  flex-direction: column;
  gap: 4px;
}
.projectName {
  color: #172033;
}
.projectDescription {
  color: #64748b;
  font-size: 0.86rem;
  line-height: 1.45;
}
.invoiceLink {
  color: #1d4ed8;
```

```
  font-weight: 700;
  text-decoration: none;
  white-space: nowrap;
}
.invoiceLink:hover {
  color: #1e40af;
  text-decoration: underline;
}
.primaryButton,
.secondaryButton {
  display: inline-flex;
  min-height: 40px;
  align-items: center;
  justify-content: center;
  border: 1px solid transparent;
  border-radius: 8px;
  cursor: pointer;
  font-weight: 700;
  padding: 0 14px;
  text-decoration: none;
  white-space: nowrap;
}
.primaryButton {
  background: #1d4ed8;
  color: #ffffff;
}
```

```
.primaryButton:hover {
  background: #1e40af;
}
.secondaryButton {
  border-color: #cbd5e1;
  background: #ffffff;
  color: #334155;
}
.secondaryButton:hover {
  background: #f8fafc;
}
.primaryButton:disabled,
.secondaryButton:disabled {
  cursor: not-allowed;
  opacity: 0.65;
}
.inlineForm {
  min-width: min(640px, calc(100vw - 64px)); /* Responsive width with a max */
  border: 1px solid #d9e1ec;
  border-radius: 8px;
  background: #f8fafc;
  padding: 16px;
}
.formGrid {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
```

```
  gap: 14px;
}
.field {
  display: flex;
  min-width: 0;
  flex-direction: column;
  gap: 6px;
  color: #526071;
  font-size: 0.82rem;
  font-weight: 700;
}
.field input,
.field select,
.field textarea {
  width: 100%;
  border: 1px solid #cbd5e1;
  border-radius: 8px;
  background: #ffffff;
  color: #172033;
  padding: 10px 12px;
}
.field textarea {
  resize: vertical;
}
.field input:focus,
.field select:focus,
```

```
.field textarea:focus {
  border-color: #1d4ed8;
  outline: 2px solid #bfdbfe;
  outline-offset: 1px;
}
.fullWidth {
  grid-column: 1 / -1; /* Spans across all columns in the grid */
}
.formActions {
  display: flex;
  flex-wrap: wrap;
  gap: 10px;
  margin-top: 14px;
}
.error {
  margin: 12px 0 0;
  border-radius: 8px;
  background: #fee2e2;
  color: #991b1b;
  font-size: 0.9rem;
  font-weight: 700;
  padding: 10px 12px;
}
.emptyState {
  display: flex;
  flex-direction: column;
```

```
  align-items: flex-start;
  gap: 16px;
  padding: 32px;
}
.emptyText {
  margin: 0;
  color: #64748b;
}
@media (max-width: 860px) {
  .header {
    flex-direction: column;
  }
  /* Converts table to a block layout for mobile */
  .table,
  .table thead,
  .table tbody,
  .table th,
  .table td,
  .table tr {
    display: block;
  }
  .table thead {
    display: none;
  }
  .table tr {
    border-bottom: 1px solid #e2e8f0;
```

```
    padding: 14px 16px;
  }
  .table tr:last-child {
    border-bottom: 0;
  }
  .table td {
    display: grid;
    grid-template-columns: minmax(120px, 42%) minmax(0, 1fr);
    gap: 12px;
    border-bottom: 0;
    padding: 8px 0;
  }
  .table td::before {
    content: attr(data-label); /* Uses data-label attribute for column headers */
    color: #64748b;
    font-size: 0.78rem;
    font-weight: 700;
    text-transform: uppercase;
  }
  .inlineForm {
    min-width: 0;
    width: 100%;
  }
}
@media (max-width: 560px) {
  .title {
```

```
  font-size: 1.65rem;
 }
 .formGrid {
  grid-template-columns: 1fr; /* Single column layout on very small screens */
 }
}
```

Header.module.css

Found in folder: **/src/components**

This CSS styles a header section with a title, subtitle, and a call-to-action button, including responsive behavior for mobile screens. It defines visual styles for layout, typography, colors, and interactive states. The media query adjusts the layout and button width on smaller screens.

```
.header {
 display: flex;
 align-items: center;
 justify-content: space-between;
 gap: 20px;
 border-bottom: 1px solid #d9e1ec;
 background: #ffffff;
 padding: 22px 32px;
}
.eyebrow {
 color: #64748b;
 font-size: 0.78rem;
 font-weight: 700;
 margin: 0 0 6px;
 text-transform: uppercase;
```

```
}
.title {
  color: #172033;
  font-size: 1.65rem;
  line-height: 1.2;
  margin: 0;
}
.action {
  display: inline-flex;
  min-height: 42px;
  align-items: center;
  justify-content: center;
  border-radius: 8px;
  background: #1d4ed8;
  color: #ffffff;
  font-weight: 700;
  padding: 0 16px;
  text-decoration: none;
  white-space: nowrap;
}
.action:hover {
  background: #1e40af; /* Darker shade on hover for visual feedback */
}
@media (max-width: 640px) {
  .header {
    align-items: flex-start;
```

```
  flex-direction: column; /* Stack header elements vertically on small screens */
  padding: 20px;
 }
 .action {
  width: 100%; /* Button spans full width on mobile */
 }
}
```

Sidebar.module.css

Found in folder: **/src/components**

This CSS styles a sidebar component with a brand logo and navigation links. It defines layout, colors, and interactive states like hover and active. On smaller screens, it adapts to a horizontal bottom-aligned navigation bar.

```
.sidebar {
 display: flex;
 min-height: 100vh; /* Full viewport height */
 flex-direction: column;
 gap: 32px; /* Spacing between child elements */
 border-right: 1px solid #d9e1ec;
 background: #f8fafc;
 padding: 24px 18px;
}
.brand {
 display: flex;
 align-items: center;
 gap: 12px;
 color: #172033;
 font-weight: 700;
```

```
  text-decoration: none;
}
.brandMark {
  display: grid;
  width: 38px;
  height: 38px;
  place-items: center; /* Centers content both horizontally and vertically */
  border-radius: 8px;
  background: #1d4ed8;
  color: #ffffff;
  font-size: 0.85rem;
  letter-spacing: 0;
}
.brandText {
  line-height: 1.2;
}
.nav {
  display: flex;
  flex-direction: column;
  gap: 6px; /* Spacing between navigation links */
}
.navLink {
  border-radius: 8px;
  color: #526071;
  font-size: 0.95rem;
  font-weight: 600;
```

```
  padding: 11px 12px;
  text-decoration: none;
  transition:
    background-color 160ms ease,
    color 160ms ease; /* Smooth hover/active state transitions */
}
.navLink:hover {
  background: #eef4ff;
  color: #1d4ed8;
}
.active {
  background: #dbeafe;
  color: #1d4ed8;
}
@media (max-width: 760px) {
  .sidebar {
    min-height: auto; /* Remove full height on mobile */
    border-right: 0;
    border-bottom: 1px solid #d9e1ec;
    padding: 18px;
  }
  .nav {
    display: grid;
    grid-template-columns: repeat(4, minmax(0, 1fr)); /* Four-column grid for links */
  }
  .navLink {
```

```
    text-align: center; /* Center text in mobile layout */
  }
}
```

StatusBadge.module.css

Found in folder: **/src/components**

This CSS code defines styles for badge components, including a base class for layout and spacing, and several color variant classes for different visual states. Each color variant sets a background and text color to create distinct appearances. The badges are designed as small, rounded, inline elements with bold text and no wrapping.

```
.badge {
  display: inline-flex;        /* Makes badge an inline flex container */
  min-height: 24px;            /* Ensures a consistent minimum height */
  align-items: center;         /* Vertically centers content */
  border-radius: 999px;        /* Creates a pill-shaped rounded border */
  font-size: 0.78rem;          /* Sets a small font size */
  font-weight: 700;            /* Makes text bold */
  line-height: 1;              /* Tight line spacing */
  padding: 5px 9px;            /* Internal spacing */
  white-space: nowrap;         /* Prevents text from wrapping */
}
/* Color variants */
.grey {
  background: #e5e7eb;
  color: #374151;
}
.blue {
  background: #dbeafe;
```

```
  color: #1d4ed8;
}
.green {
  background: #dcfce7;
  color: #15803d;
}
.red {
  background: #fee2e2;
  color: #b91c1c;
}
.yellow {
  background: #fef3c7;
  color: #a16207;
}
.darkGrey {
  background: #374151;
  color: #f9fafb;
}
```

clientActions.js

Found in folder: **/src/actions**

This code defines server actions for creating, updating, and deleting client records. It validates input data, converts FormData to objects, and handles errors with user-friendly messages. The actions wrap service functions and return standardized result objects.

```
'use server'
import {
  createClient,
  deleteClient,
```

```
  updateClient,
} from '../lib/services.js'
// Helper function to return standardized result objects
function result(data = null, error = null) {
  return { data, error }
}
// Extract meaningful error message from error objects
function errorMessage(error) {
  return error instanceof Error ? error.message : 'Something went wrong'
}
// Convert FormData to plain object if input is FormData
function formDataToObject(input) {
  if (!(input instanceof FormData)) {
    return input
  }
  return Object.fromEntries(input.entries())
}
export async function createClientAction(input) {
  const data = formDataToObject(input)
  // Validate required client name field
  if (!data?.name || String(data.name).trim() === '') {
    return result(null, 'Client name is required')
  }
  try {
    return result(createClient(data), null)
  } catch (error) {
```

```
    return result(null, errorMessage(error))
  }
}
export async function updateClientAction(id, data) {
  try {
    return result(updateClient(id, formDataToObject(data)), null)
  } catch (error) {
    return result(null, errorMessage(error))
  }
}
export async function deleteClientAction(id) {
  try {
    deleteClient(id)
    return result(true, null)
  } catch (error) {
    const message = errorMessage(error)
    // Handle specific constraint error for clients with projects
    if (message.includes('because it has projects')) {
      return result(
        null,
        'Cannot delete this client because projects are still assigned to it.',
      )
    }
    return result(null, message)
  }
}
```

invoiceActions.js

Found in folder: **/src/actions**

This code defines server-side actions for managing invoices, including creating, updating, deleting, and updating invoice statuses. It validates input data for creating invoices and ensures status updates are valid before applying changes. Each action returns a standardized result object containing either data or an error message.

```
'use server'

import {
  createInvoice,
  deleteInvoice,
  getInvoiceById,
  updateInvoice,
} from '../lib/services.js'

// Valid invoice statuses for status update validation
const INVOICE_STATUSES = new Set(['draft', 'sent', 'paid', 'overdue', 'void'])

// Helper function to standardize response format
function result(data = null, error = null) {
  return { data, error }
}

// Helper to extract error message from caught errors
function errorMessage(error) {
  return error instanceof Error ? error.message : 'Something went wrong'
}

export async function createInvoiceAction(data) {
  // Validate required fields for invoice creation
  if (!data?.projectId || String(data.projectId).trim() === '') {
    return result(null, 'Project is required')
```

```
  }
  if (!data?.invoiceNumber || String(data.invoiceNumber).trim() === '') {
   return result(null, 'Invoice number is required')
  }
  if (!data?.issueDate || String(data.issueDate).trim() === '') {
   return result(null, 'Issue date is required')
  }
  if (!Array.isArray(data.lineItems) || data.lineItems.length === 0) {
   return result(null, 'At least one invoice line item is required')
  }
  try {
   return result(createInvoice(data), null)
  } catch (error) {
   return result(null, errorMessage(error))
  }
}
export async function updateInvoiceAction(id, data) {
  try {
   return result(updateInvoice(id, data), null)
  } catch (error) {
   return result(null, errorMessage(error))
  }
}
export async function deleteInvoiceAction(id) {
  try {
   deleteInvoice(id)
```

```
    return result(true, null)
  } catch (error) {
    return result(null, errorMessage(error))
  }
}
export async function updateInvoiceStatusAction(id, status) {
  // Validate that the provided status is allowed
  if (!INVOICE_STATUSES.has(status)) {
    return result(null, `Invalid invoice status: ${status}`)
  }
  try {
    const invoice = getInvoiceById(id)
    if (!invoice) {
      return result(null, `Invoice ${id} not found`)
    }
    // Update paid amount and date only when status changes to 'paid'
    const nextInvoice = {
      ...invoice,
      status,
      paidCents: status === 'paid' ? invoice.totalCents : invoice.paidCents,
      paidAt: status === 'paid' ? (invoice.paidAt || new Date().toISOString()) : invoice.paidAt,
    }
    return result(updateInvoice(id, nextInvoice), null)
  } catch (error) {
    return result(null, errorMessage(error))
  }
```

```
}
```

projectActions.js

Found in folder: **/src/actions**

This code defines server actions for creating, updating, and deleting projects. It validates input for the create action and handles errors by returning user-friendly messages. The delete action includes a specific check for projects that cannot be deleted due to existing invoices.

```
'use server'

import {
  createProject,
  deleteProject,
  updateProject,
} from '../lib/services.js'

// Helper function to standardize the response format for all actions
function result(data = null, error = null) {
  return { data, error }
}

// Helper to extract a safe error message from caught errors
function errorMessage(error) {
  return error instanceof Error ? error.message : 'Something went wrong'
}

export async function createProjectAction(data) {
  // Validate required fields for project creation
  if (!data?.clientId || String(data.clientId).trim() === '') {
    return result(null, 'Client is required')
  }
  if (!data?.name || String(data.name).trim() === '') {
```

```
    return result(null, 'Project name is required')
  }
  try {
    return result(createProject(data), null)
  } catch (error) {
    return result(null, errorMessage(error))
  }
}
export async function updateProjectAction(id, data) {
  try {
    return result(updateProject(id, data), null)
  } catch (error) {
    return result(null, errorMessage(error))
  }
}
export async function deleteProjectAction(id) {
  try {
    deleteProject(id)
    return result(true, null)
  } catch (error) {
    const message = errorMessage(error)
    // Provide a specific error message if deletion fails due to existing invoices
    if (message.includes('because it has invoices')) {
      return result(
        null,
        'Cannot delete this project because invoices are still assigned to it.',
```

```
    )
  }
  return result(null, message)
 }
}
```

error.js

Found in folder: **/src/app**

This code defines a React component that displays an error message with a refresh button. When clicked, the button triggers a page refresh using Next.js's router. The component is styled with CSS modules and includes ARIA roles for accessibility.

```
"use client";

import { useRouter } from "next/navigation";

import styles from "./Error.module.css";

export default function Error() {
 const router = useRouter();
 return (
  <section className={styles.errorState} role="alert">
   <p className={styles.message}>
    Something went wrong. Please try refreshing the page.
   </p>
   <button
    className={styles.button}
    onClick={() => router.refresh()} // Refreshes the current page
    type="button"
   >
    Refresh
   </button>
```

```
  </section>
 );
}
```

layout.js

Found in folder: **/src/app**

This code defines the root layout for the application, setting global metadata and structuring the page with a sidebar, header, and main content area. It imports and applies CSS styles to arrange these components. It also injects an external script from a local development server.

```
import "./globals.css";
import Header from "@/components/Header";
import Sidebar from "@/components/Sidebar";
import styles from "./layout.module.css";
// Metadata for the application
export const metadata = {
 title: "Invoice Tracker",
 description: "Freelancer invoice tracker",
};
// Root layout component defining the overall page structure
export default function RootLayout({ children }) {
 return (
  <html lang="en">
   <body>
    {/* Container for the sidebar and main content area */}
    <div className={styles.shell}>
     <Sidebar />
     <div className={styles.content}>
```

```
        <Header />
        {/* Main area where page content is rendered */}
        <main className={styles.main}>{children}</main>
       </div>
      </div>
     {/* Script injected for development purposes (e.g., live reload or iframe communication) */}
      <Script src="http://localhost:5000/iframe-script" />
    </body>
   </html>
 );
}
```

page.js

Found in folder: **/src/app**

This code renders a dashboard page that displays business summary cards, a revenue chart for the last six months, and a list of overdue invoices. It fetches summary data, monthly revenue, and overdue invoices, then formats and presents them using styled components. If there are no overdue invoices, it shows a message instead of a list.

```
import Link from "next/link";
import { formatCurrency, formatDate, formatInvoiceNumber } from "@/lib/formatters";
import {
  getDashboardSummary,
  getMonthlyRevenue,
  getOverdueInvoices,
} from "@/lib/services";
import RevenueChart from "./RevenueChart";
import styles from "./Dashboard.module.css";
```

```
export default function DashboardPage() {
 // Fetch data for dashboard display
 const summary = getDashboardSummary();
 const monthlyRevenue = getMonthlyRevenue();
 const overdueInvoices = getOverdueInvoices();
 return (
  <section className={styles.page}>
   <div className={styles.header}>
    <div>
     <p className={styles.eyebrow}>Dashboard</p>
     <h1 className={styles.title}>Business overview</h1>
    </div>
   </div>
   {/* Summary cards showing key business metrics */}
   <div className={styles.summaryGrid}>
    <article className={styles.summaryCard}>
     <p className={styles.cardLabel}>Total Outstanding</p>
     <p className={styles.cardValue}>
      {formatCurrency(summary.totalOutstandingCents)}
     </p>
    </article>
    <article className={styles.summaryCard}>
     <p className={styles.cardLabel}>Paid This Month</p>
     <p className={styles.cardValue}>
      {formatCurrency(summary.totalPaidThisMonthCents)}
     </p>
```

```
  </article>
  <article className={styles.summaryCard}>
   <p className={styles.cardLabel}>Overdue Invoices</p>
   <p
    className={` ${styles.cardValue} ${
     summary.overdueCount > 0 ? styles.dangerValue : ""
    }`}
   >
    {summary.overdueCount}
   </p>
  </article>
  <article className={styles.summaryCard}>
   <p className={styles.cardLabel}>Top Client</p>
   <p className={styles.cardValue}>
    {summary.topClientByRevenue || "No paid invoices"}
   </p>
  </article>
 </div>
 {/* Revenue chart section for the last six months */}
 <section className={styles.panel} aria-labelledby="revenue-heading">
  <div className={styles.panelHeader}>
   <div>
    <p className={styles.eyebrow}>Revenue</p>
    <h2 id="revenue-heading" className={styles.sectionTitle}>
     Last 6 months
    </h2>
```

```
  </div>
 </div>
 <RevenueChart data={monthlyRevenue} />
</section>
{/* Overdue invoices list or empty state message */}
<section className={styles.panel} aria-labelledby="overdue-heading">
 <div className={styles.panelHeader}>
  <div>
   <p className={styles.eyebrow}>Invoices</p>
   <h2 id="overdue-heading" className={styles.sectionTitle}>
    Overdue invoices
   </h2>
  </div>
 </div>
 {overdueInvoices.length === 0 ? (
  <div className={styles.emptyState}>
   <h3 className={styles.emptyTitle}>No overdue invoices</h3>
   <p className={styles.emptyText}>
    Everything is current. There are no overdue balances to chase.
   </p>
  </div>
 ) : (
  <div className={styles.overdueList}>
   {overdueInvoices.map((invoice) => {
    const amountDueCents = invoice.totalCents - invoice.paidCents;
    return (
```

```
            <Link
              key={invoice.id}
              href={`/invoices/${invoice.id}`}
              className={styles.overdueRow}
            >
              <div className={styles.invoiceIdentity}>
                <span className={styles.clientName}>{invoice.clientName}</span>
                <span className={styles.invoiceNumber}>
                  {formatInvoiceNumber(invoice.invoiceNumber)}
                </span>
              </div>
              <div className={styles.invoiceMeta}>
                <span>{formatDate(invoice.dueDate)}</span>
                <strong>{formatCurrency(amountDueCents, invoice.currency)}</strong>
              </div>
            </Link>
          );
        })}
      </div>
    )}
  </section>
 </section>
);
}
```

route.js

Found in folder: **/src/app/api/invoices/export**

This code defines an API endpoint that exports invoice data as a CSV file. It filters invoices based on URL query parameters, formats the data appropriately, and returns the CSV as a downloadable file with proper headers.

```
import { getInvoices } from "@/lib/services";

// Column headers for the CSV output

const csvColumns = [

  "Invoice Number",

  "Client",

  "Project",

  "Issue Date",

  "Due Date",

  "Status",

  "Total",

];

// Extracts and validates filter values from URL search parameters

function getFilter(searchParams, name) {

  const value = searchParams.get(name);

  if (value === null || value === "" || value === "all") {

    return undefined;

  }

  return value;

}

// Formats date values to YYYY-MM-DD format for CSV

function formatCsvDate(value) {

  if (!value) {

    return "";

  }
```

```
  return String(value).slice(0, 10);
}
// Converts cents to dollars with two decimal places
function formatCsvTotal(cents) {
  return (Number(cents || 0) / 100).toFixed(2);
}
// Escapes CSV special characters and wraps fields in quotes when needed
function escapeCsvField(value) {
  const text = String(value ?? "");
  if (/[",\r\n]/.test(text)) {
    return `"${text.replaceAll('"', '""')}"`;
  }
  return text;
}
// Converts 2D array of data into CSV string format
function serializeCsv(rows) {
  return rows
    .map((row) => row.map(escapeCsvField).join(","))
    .join("\r\n");
}
// API endpoint handler for CSV export
export async function GET(request) {
  const { searchParams } = new URL(request.url);
  const invoices = getInvoices({
    status: getFilter(searchParams, "status"),
    clientId: getFilter(searchParams, "clientId"),
```

```
  projectId: getFilter(searchParams, "projectId"),
 });
 // Build CSV rows: headers first, then invoice data
 const rows = [
  csvColumns,
  ...invoices.map((invoice) => [
   invoice.invoiceNumber,
   invoice.clientName,
   invoice.projectName,
   formatCsvDate(invoice.issueDate),
   formatCsvDate(invoice.dueDate),
   invoice.status,
   formatCsvTotal(invoice.totalCents),
  ]),
 ];
 // Return CSV response with download headers
 return new Response(`${serializeCsv(rows)}\r\n`, {
  headers: {
   "Content-Type": "text/csv",
   "Content-Disposition": 'attachment; filename="invoices.csv"',
  },
 });
}
```

error.js

Found in folder: **/src/app/clients**

This component renders an error state UI when client data fails to load, displaying a message and a retry button. The button triggers a retry function passed as a prop to attempt reloading the data. It uses CSS modules for styling and includes ARIA roles for accessibility.

```
"use client";

import styles from "./Error.module.css";

export default function ClientsError({ unstable_retry }) {
  return (
    <section className={styles.errorState} role="alert">
      <p className={styles.message}>
        Unable to load clients. Please try again.
      </p>
      <button
        className={styles.button}
        onClick={() => unstable_retry()} // Calls the retry function passed via props
        type="button"
      >
        Try Again
      </button>
    </section>
  );
}
```

page.js

Found in folder: **/src/app/clients**

This component displays a list of clients. It shows a header with a form to create a new client and a main panel. The panel either shows an empty state message with a creation form if there are no clients, or a table listing all clients with their details and an edit form.

```
import StatusBadge from "@/components/StatusBadge";
import { formatCurrency } from "@/lib/formatters";
import { getClients } from "@/lib/services";
import ClientInlineForm from "./ClientInlineForm";
import styles from "./ClientsPage.module.css";
export default function ClientsPage() {
 // Fetch the list of clients from the service layer
 const clients = getClients();
 return (
  <section className={styles.page}>
   <div className={styles.header}>
    <div>
     <p className={styles.eyebrow}>Clients</p>
     <h1 className={styles.title}>Client accounts</h1>
    </div>
    {/* Form for creating a new client directly in the header */}
    <ClientInlineForm mode="create" />
   </div>
   <div className={styles.panel}>
    {/* Conditional rendering: empty state vs. client table */}
    {clients.length === 0 ? (
     <div className={styles.emptyState}>
      <p className={styles.emptyText}>
       No clients yet. Add your first client to get started.
      </p>
      <ClientInlineForm mode="create" />
```

```
  </div>
) : (
  <div className={styles.tableWrap}>
    <table className={styles.table}>
      <thead>
        <tr>
          <th scope="col">Name</th>
          <th scope="col">Company</th>
          <th scope="col">Email</th>
          <th scope="col">Active projects</th>
          <th scope="col">Unpaid</th>
          <th scope="col">Actions</th>
        </tr>
      </thead>
      <tbody>
        {/* Map over each client to create a table row */}
        {clients.map((client) => (
          <tr key={client.id}>
            <td data-label="Name">
              <strong className={styles.clientName}>{client.name}</strong>
            </td>
            <td data-label="Company">{client.company || "—"}</td>
            <td data-label="Email">{client.email || "—"}</td>
            <td data-label="Active projects">
              <span className={styles.projectCount}>
                <span>{client.activeProjectCount}</span>
```

```
                <StatusBadge status="active" />
              </span>
            </td>
            <td data-label="Unpaid">
              {/* Format currency from cents to a readable string */}
              {formatCurrency(client.totalUnpaidCents)}
            </td>
            <td data-label="Actions">
              {/* Inline form for editing this specific client */}
              <ClientInlineForm mode="edit" client={client} />
            </td>
          </tr>
        ))}
      </tbody>
    </table>
  </div>
 )}
 </div>
 </section>
 );
}
```

error.js

Found in folder: **/src/app/invoices**

This code defines a React error boundary component that displays a user-friendly error message and a retry button when invoices fail to load. It provides visual feedback through CSS modules and includes accessibility features like role=”alert”. The component accepts

a retry function prop that gets called when the user clicks the button to attempt reloading the data.

```
"use client";

import styles from "./Error.module.css";

// Error boundary component for invoice loading failures

export default function InvoicesError({ unstable_retry }) {

 return (

  <section className={styles.errorState} role="alert">

   <p className={styles.message}>

    Unable to load invoices. Please try again.

   </p>

   <button

    className={styles.button}

    onClick={() => unstable_retry()}  // Retry function passed as prop

    type="button"

   >

    Try Again

   </button>

  </section>

 );

}
```

page.js

Found in folder: **/src/app/invoices**

This code renders a page that displays invoices, applying optional filters from URL search parameters. It fetches invoices using these filters and passes them to a table component for display. The page also shows a header with a title and checks if any invoices exist.

```
import { getInvoices } from "@/lib/services";
```

```
import InvoicesTable from "./InvoicesTable";
import styles from "./InvoicesPage.module.css";
// Extracts and validates a single search parameter value
function getSearchParam(searchParams, name) {
  const value = searchParams?.[name];
  const stringValue = Array.isArray(value) ? value[0] : value;
  // Return undefined for empty values or the "all" placeholder
  if (!stringValue || stringValue === "all") {
    return undefined;
  }
  return stringValue;
}
export default async function InvoicesPage({ searchParams }) {
  const resolvedSearchParams = await searchParams;
  // Build filter object from URL parameters
  const filters = {
    status: getSearchParam(resolvedSearchParams, "status"),
    clientId: getSearchParam(resolvedSearchParams, "clientId"),
    projectId: getSearchParam(resolvedSearchParams, "projectId"),
  };
  const invoices = getInvoices(filters);
  // Check if any invoices exist (without filters)
  const hasInvoices = getInvoices().length > 0;
  return (
    <section className={styles.page}>
      <div className={styles.header}>
```

```
      <div>
        <p className={styles.eyebrow}>Invoices</p>
        <h1 className={styles.title}>Invoice ledger</h1>
      </div>
    </div>
    <InvoicesTable
      hasInvoices={hasInvoices}
      invoices={invoices}
      initialFilters={filters}
    />
  </section>
 );
}
```

page.js

Found in folder: **/src/app/invoices/new**

This code renders a page for creating a new invoice. It fetches client and project data, then passes it to an invoice form component set to creation mode. The page includes a styled header section with a title.

```
import { getClients, getProjects } from "@/lib/services";
import InvoiceForm from "../InvoiceForm";
import styles from "../InvoicesPage.module.css";
export default function NewInvoicePage() {
 // Fetch data needed for the invoice form
 const clients = getClients();
 const projects = getProjects();
 return (
  <section className={styles.page}>
```

```
    <div className={styles.header}>
     <div>
      <p className={styles.eyebrow}>Invoices</p>
      <h1 className={styles.title}>Create invoice</h1>
     </div>
    </div>
    {/* Form component pre-populated with available clients and projects */}
    <InvoiceForm mode="create" clients={clients} projects={projects} />
   </section>
  );
}
```

page.js

Found in folder: **/src/app/invoices/[invoiceId]**

This code renders an invoice detail page with actions to update its status, displays invoice information like client details and line items, and includes validation for the invoice ID and status transitions. It formats quantities and currency values appropriately and conditionally shows action buttons based on the current invoice status. The page also revalidates cache paths after status updates to ensure data freshness.

```
import Link from "next/link";
import { revalidatePath } from "next/cache";
import { notFound } from "next/navigation";
import { updateInvoiceStatusAction } from "@/actions/invoiceActions";
import StatusBadge from "@/components/StatusBadge";
import {
 formatCurrency,
 formatDate,
 formatInvoiceNumber,
```

```
} from "@/lib/formatters";

import { getInvoiceById } from "@/lib/services";

import styles from "./InvoiceDetailPage.module.css";

// Formats quantity values, preserving non-numeric strings and showing two decimals for non-integers

function formatQuantity(quantity) {

  const value = Number(quantity);

  if (!Number.isFinite(value)) {

    return quantity;

  }

  return Number.isInteger(value) ? String(value) : value.toFixed(2);

}

// Conditionally renders a contact line only if children content exists

function ContactLine({ children }) {

  if (!children) {

    return null;

  }

  return <p className={styles.contactLine}>{children}</p>;

}

// Validates that the value is a positive integer (no leading zeros)

function isPositiveInteger(value) {

  return /^[1-9]\d*$/.test(String(value));

}

// Creates a server action to update invoice status and revalidate relevant paths

function createStatusAction(invoiceId, status) {

  return async function updateStatus() {
```

```
    "use server";
    const result = await updateInvoiceStatusAction(invoiceId, status);
    if (result.error) {
      throw new Error(result.error);
    }
    revalidatePath("/invoices");
    revalidatePath(`/invoices/${invoiceId}`);
  };
}
export default async function InvoiceDetailPage({ params }) {
  const { invoiceId } = await params;
  // Validate invoice ID format before fetching
  if (!isPositiveInteger(invoiceId)) {
    notFound();
  }
  const invoice = getInvoiceById(invoiceId);
  if (!invoice) {
    notFound();
  }
  const invoiceNumber = formatInvoiceNumber(invoice.invoiceNumber);
  const markSentAction = createStatusAction(invoice.id, "sent");
  const markPaidAction = createStatusAction(invoice.id, "paid");
  const voidAction = createStatusAction(invoice.id, "void");
  const canMarkSent = invoice.status === "draft";
  const canMarkPaid = invoice.status === "sent" || invoice.status === "overdue";
  const canVoid = invoice.status !== "void" && invoice.status !== "paid";
```

```
const balanceDueCents = Math.max(
  invoice.totalCents - invoice.paidCents,
  0,
);
return (
  <section className={styles.page}>
    <div className={styles.actions} aria-label="Invoice actions">
      <Link className={styles.secondaryLink} href="/invoices">
        Back to Invoices
      </Link>
      {canMarkSent ? (
        <form action={markSentAction}>
          <button className={styles.secondaryButton} type="submit">
            Mark as Sent
          </button>
        </form>
      ) : null}
      {canMarkPaid ? (
        <form action={markPaidAction}>
          <button className={styles.primaryButton} type="submit">
            Mark as Paid
          </button>
        </form>
      ) : null}
      {canVoid ? (
        <form action={voidAction}>
```

```
    <button className={styles.dangerButton} type="submit">
      Void
    </button>
  </form>
 ) : null}
 <Link
  className={styles.primaryLink}
  href={`/invoices/${invoice.id}/edit`}
 >
  Edit
 </Link>
</div>
<article className={styles.document}>
 <header className={styles.documentHeader}>
  <div>
   <p className={styles.brandLabel}>Freelancer Invoice</p>
   <p className={styles.tagline}>Professional Services</p>
  </div>
  <div className={styles.invoiceMeta}>
   <p className={styles.invoiceLabel}>Invoice</p>
   <h1 className={styles.invoiceNumber}>{invoiceNumber}</h1>
   <dl className={styles.metaList}>
    <div>
     <dt>Issue date</dt>
     <dd>{formatDate(invoice.issueDate)}</dd>
    </div>
```

```
    <div>
      <dt>Due date</dt>
      <dd>{formatDate(invoice.dueDate)}</dd>
    </div>
    <div>
      <dt>Status</dt>
      <dd>
        <StatusBadge status={invoice.status} />
      </dd>
    </div>
  </dl>
 </div>
</header>
<div className={styles.parties}>
 <section className={styles.billTo} aria-labelledby="bill-to-heading">
  <h2 id="bill-to-heading">Bill To:</h2>
  <div className={styles.clientBlock}>
    <p className={styles.clientName}>{invoice.client.name}</p>
    <ContactLine>{invoice.client.company}</ContactLine>
    <ContactLine>{invoice.client.email}</ContactLine>
    {invoice.client.address ? (
      <p className={styles.address}>{invoice.client.address}</p>
    ) : null}
  </div>
 </section>
 <section className={styles.project} aria-labelledby="project-heading">
```

```
  <h2 id="project-heading">Project</h2>
  <p className={styles.projectName}>{invoice.project.name}</p>
  <p className={styles.projectNote}>
   This invoice covers work completed for {invoice.project.name}.
  </p>
 </section>
</div>
<section className={styles.lineItems} aria-labelledby="items-heading">
 <h2 id="items-heading" className={styles.sectionTitle}>
  Services
 </h2>
 <div className={styles.tableWrap}>
  <table className={styles.table}>
   <thead>
    <tr>
     <th scope="col">Description</th>
     <th scope="col">Quantity</th>
     <th scope="col">Unit Price</th>
     <th scope="col">Total</th>
    </tr>
   </thead>
   <tbody>
    {invoice.lineItems.map((item) => (
     <tr key={item.id}>
      <td>{item.description}</td>
      <td>{formatQuantity(item.quantity)}</td>
```

```
          <td>
            {formatCurrency(item.unitPriceCents, invoice.currency)}
          </td>
          <td>
            {formatCurrency(item.lineTotalCents, invoice.currency)}
          </td>
        </tr>
      ))}
    </tbody>
    <tfoot>
      <tr>
        <th scope="row" colSpan="3">
          Subtotal
        </th>
        <td>{formatCurrency(invoice.subtotalCents, invoice.currency)}</td>
      </tr>
      <tr className={styles.totalRow}>
        <th scope="row" colSpan="3">
          Total
        </th>
        <td>{formatCurrency(invoice.totalCents, invoice.currency)}</td>
      </tr>
    </tfoot>
  </table>
 </div>
</section>
```

```
      <section className={styles.payment} aria-labelledby="payment-heading">
       <div>
        <h2 id="payment-heading">Payment</h2>
        <p className={styles.paymentDueDate}>
         Payment due by {formatDate(invoice.dueDate)}
        </p>
       </div>
       <div className={styles.amountDue}>
        <p>Total amount due</p>
        <strong>{formatCurrency(balanceDueCents, invoice.currency)}</strong>
       </div>
      </section>
      {invoice.notes ? (
       <section className={styles.notes} aria-labelledby="notes-heading">
        <h2 id="notes-heading">Notes</h2>
        <p>{invoice.notes}</p>
       </section>
      ) : null}
     </article>
    </section>
  );
}
```

page.js

Found in folder: **/src/app/invoices/[invoiceId]/edit**

This code loads an invoice by ID and renders a form to edit it, fetching related clients and projects for the form. If the invoice doesn't exist, it shows a 404 page. It then passes all this data to the InvoiceForm component in edit mode.

```
import { notFound } from "next/navigation";
import { getClients, getInvoiceById, getProjects } from "@/lib/services";
import InvoiceForm from "../../InvoiceForm";
import styles from "../../InvoicesPage.module.css";
export default async function EditInvoicePage({ params }) {
 // Extract invoice ID from dynamic route parameters
 const { invoiceId } = await params;
 const invoice = getInvoiceById(invoiceId);
 // Show 404 page if invoice doesn't exist
 if (!invoice) {
  notFound();
 }
 // Fetch all clients and projects for form dropdowns
 const clients = getClients();
 const projects = getProjects();
 return (
  <section className={styles.page}>
   <div className={styles.header}>
    <div>
     <p className={styles.eyebrow}>Invoices</p>
     <h1 className={styles.title}>Edit invoice</h1>
    </div>
   </div>
   {/* Pass invoice data and related entities to form component */}
   <InvoiceForm
    mode="edit"
```

```
      invoice={invoice}
      clients={clients}
      projects={projects}
    />
  </section>
 );
}
```

error.js

Found in folder: **/src/app/projects**

This code defines a React error boundary component that displays a user-friendly error message and a retry button when projects fail to load. It renders a styled alert section containing an error message and a button that triggers the provided retry function when clicked. The component uses CSS modules for styling and is marked as a client component for Next.js.

```
"use client";

import styles from "./Error.module.css";

export default function ProjectsError({ unstable_retry }) {
 return (
  <section className={styles.errorState} role="alert">
   <p className={styles.message}>
    Unable to load projects. Please try again.
   </p>
   <button
    className={styles.button}
    onClick={() => unstable_retry()} // Triggers retry mechanism when clicked
    type="button"
   >
```

```
      Try Again
    </button>
  </section>
 );
}
```

page.js

Found in folder: **/src/app/projects**

This code displays a list of projects grouped by their client, showing each project's name, status, total invoiced amount, and a link to its invoices. It provides an inline form to add new projects and shows an empty state message when no projects exist. The projects are sorted alphabetically by client name.

```
import Link from "next/link";
import StatusBadge from "@/components/StatusBadge";
import { formatCurrency } from "@/lib/formatters";
import { getClients, getProjects } from "@/lib/services";
import ProjectInlineForm from "./ProjectInlineForm";
import styles from "./ProjectsPage.module.css";
// Groups an array of projects by their client, returning sorted groups
function groupProjectsByClient(projects) {
 const groups = new Map();
 for (const project of projects) {
  const group = groups.get(project.clientId) || {
   clientId: project.clientId,
   clientName: project.clientName,
   projects: [],
  };
  group.projects.push(project);
```

```
    groups.set(project.clientId, group);
  }
  // Convert map to array and sort groups alphabetically by client name
  return Array.from(groups.values()).sort((first, second) => (
    first.clientName.localeCompare(second.clientName)
  ));
}
export default function ProjectsPage() {
  const projects = getProjects();
  const clients = getClients();
  const projectGroups = groupProjectsByClient(projects);
  return (
    <section className={styles.page}>
      <div className={styles.header}>
        <div>
          <p className={styles.eyebrow}>Projects</p>
          <h1 className={styles.title}>Project ledger</h1>
        </div>
        <ProjectInlineForm clients={clients} />
      </div>
      <div className={styles.panel}>
        {projectGroups.length === 0 ? (
          // Show empty state with form to add first project
          <div className={styles.emptyState}>
            <p className={styles.emptyText}>
              No projects yet. Add your first project to get started.
```

```
  </p>
  <ProjectInlineForm clients={clients} />
 </div>
) : (
 // Render project groups, each with a table of projects
 <div className={styles.clientSections}>
  {projectGroups.map((group) => (
   <section className={styles.clientSection} key={group.clientId}>
    <header className={styles.clientHeader}>
     <h2 className={styles.clientName}>{group.clientName}</h2>
    </header>
    <div className={styles.tableWrap}>
     <table className={styles.table}>
      <thead>
       <tr>
        <th scope="col">Project</th>
        <th scope="col">Status</th>
        <th scope="col">Total invoiced</th>
        <th scope="col">Invoices</th>
       </tr>
      </thead>
      <tbody>
       {group.projects.map((project) => (
        <tr key={project.id}>
         <td data-label="Project">
          <div className={styles.projectSummary}>
```

```
      <strong className={styles.projectName}>
       {project.name}
      </strong>
      {project.description ? (
       <span className={styles.projectDescription}>
        {project.description}
       </span>
      ) : null}
     </div>
    </td>
    <td data-label="Status">
     <StatusBadge status={project.status} />
    </td>
    <td data-label="Total invoiced">
     {formatCurrency(project.totalInvoicedCents)}
    </td>
    <td data-label="Invoices">
     <Link
      className={styles.invoiceLink}
      href={`/invoices?projectId=${project.id}`}
     >
      View invoices
     </Link>
    </td>
   </tr>
  ))}
```

```
            </tbody>
          </table>
        </div>
      </section>
    ))}
  </div>
 )}
 </div>
</section>
);
}
```

db.js

Found in folder: **/src/lib**

This code initializes a SQLite database connection, setting the database file path based on the environment (in-memory for testing, file-based otherwise). It configures the database to use Write-Ahead Logging (WAL) mode and enables foreign key constraints. Finally, it exports the database connection instance for use elsewhere.

```
import Database from 'better-sqlite3'

import path from 'node:path'

// Determine database path: in-memory for tests, file-based for other environments

const dbPath = process.env.NODE_ENV === 'test'
 ? ':memory:'
 : path.join(process.cwd(), 'invoice-tracker.db')

const db = new Database(dbPath)

// Configure database for better performance and enforce referential integrity

db.pragma('journal_mode = WAL')

db.pragma('foreign_keys = ON')
```

export default db

formatters.js

Found in folder: **/src/lib**

This code provides three utility functions: one converts cents to a localized currency string, another formats an ISO date string into a UK-style date, and a third pads numeric invoice numbers with leading zeros.

```
export function formatCurrency(cents, currency = "USD") {
 // Convert cents to a decimal amount (e.g., 1999 cents -> 19.99)
 const amount = Number(cents || 0) / 100;
 // Format the amount as currency using the browser's Intl API
 return new Intl.NumberFormat("en-US", {
  style: "currency",
  currency,
 }).format(amount);
}
export function formatDate(isoString) {
 // Return empty string for falsy input
 if (!isoString) {
  return "";
 }
 // Format date in UK style (DD Mon YYYY) using UTC timezone
 return new Intl.DateTimeFormat("en-GB", {
  day: "2-digit",
  month: "short",
  year: "numeric",
  timeZone: "UTC",
 }).format(new Date(isoString));
```

```
}

export function formatInvoiceNumber(number) {
  const value = String(number);
  // Pad numeric strings with leading zeros to 4 digits
  if (/^\d+$/.test(value)) {
    return value.padStart(4, "0");
  }
  // Return non-numeric strings unchanged
  return value;
}
```

migrate.js

Found in folder: **/src/lib**

This code defines a database migration that creates tables for clients, projects, invoices, and invoice line items with their respective indexes and constraints. It ensures the tables only exist if they haven't been created already, using SQLite's transaction and exec methods. The migration runs immediately when the module is loaded.

```
import db from './db.js'

// Define and execute a database migration transaction
const migrate = db.transaction(() => {
  db.exec(`
    -- Create clients table with contact information and timestamps
    CREATE TABLE IF NOT EXISTS clients (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      name TEXT NOT NULL,
      email TEXT,
      phone TEXT,
      company TEXT,
```

```
  address TEXT,
  notes TEXT,
  created_at TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ', 'now')),
  updated_at TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ', 'now'))
);
-- Index for faster client name lookups
CREATE INDEX IF NOT EXISTS idx_clients_name
  ON clients (name);
-- Create projects table linked to clients with status and rate constraints
CREATE TABLE IF NOT EXISTS projects (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  client_id INTEGER NOT NULL,
  name TEXT NOT NULL,
  description TEXT,
  status TEXT NOT NULL DEFAULT 'active',
  hourly_rate_cents INTEGER,
  created_at TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ', 'now')),
  updated_at TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ', 'now')),
  FOREIGN KEY (client_id) REFERENCES clients (id) ON DELETE RESTRICT,
  CHECK (status IN ('active', 'paused', 'completed', 'archived')),
  CHECK (hourly_rate_cents IS NULL OR hourly_rate_cents >= 0)
);
-- Indexes for project filtering by client and status
CREATE INDEX IF NOT EXISTS idx_projects_client_id
  ON projects (client_id);
CREATE INDEX IF NOT EXISTS idx_projects_status
```

```
  ON projects (status);
-- Create invoices table with financial tracking and validation rules
CREATE TABLE IF NOT EXISTS invoices (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  project_id INTEGER NOT NULL,
  invoice_number TEXT NOT NULL UNIQUE,
  status TEXT NOT NULL DEFAULT 'draft',
  issue_date TEXT NOT NULL,
  due_date TEXT,
  currency TEXT NOT NULL DEFAULT 'USD',
  subtotal_cents INTEGER NOT NULL DEFAULT 0,
  total_cents INTEGER NOT NULL DEFAULT 0,
  paid_cents INTEGER NOT NULL DEFAULT 0,
  sent_at TEXT,
  paid_at TEXT,
  notes TEXT,
  created_at TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ', 'now')),
  updated_at TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ', 'now')),
  FOREIGN KEY (project_id) REFERENCES projects (id) ON DELETE RESTRICT,
  CHECK (status IN ('draft', 'sent', 'paid', 'overdue', 'void')),
  CHECK (length(currency) = 3 AND currency = upper(currency)),
  CHECK (subtotal_cents >= 0),
  CHECK (total_cents >= 0),
  CHECK (paid_cents >= 0),
  CHECK (paid_cents <= total_cents)
);
```

```
-- Indexes for invoice filtering and date-based queries
CREATE INDEX IF NOT EXISTS idx_invoices_project_id
  ON invoices (project_id);
CREATE INDEX IF NOT EXISTS idx_invoices_status
  ON invoices (status);
CREATE INDEX IF NOT EXISTS idx_invoices_due_date
  ON invoices (due_date);
CREATE INDEX IF NOT EXISTS idx_invoices_issue_date
  ON invoices (issue_date);
-- Create line items table for invoice details with position ordering
CREATE TABLE IF NOT EXISTS invoice_line_items (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  invoice_id INTEGER NOT NULL,
  position INTEGER NOT NULL,
  description TEXT NOT NULL,
  quantity REAL NOT NULL,
  unit_price_cents INTEGER NOT NULL,
  line_total_cents INTEGER NOT NULL,
  created_at TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ', 'now')),
  updated_at TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ', 'now')),
  FOREIGN KEY (invoice_id) REFERENCES invoices (id) ON DELETE CASCADE,
  UNIQUE (invoice_id, position),
  CHECK (position >= 0),
  CHECK (quantity > 0),
  CHECK (unit_price_cents >= 0),
  CHECK (line_total_cents >= 0)
```

```
  );
  -- Index for faster line item retrieval by invoice
  CREATE INDEX IF NOT EXISTS idx_invoice_line_items_invoice_id
    ON invoice_line_items (invoice_id);
  `)
})
// Execute the migration immediately
migrate()
```

seed.js

Found in folder: **/src/lib**

This code seeds a database with sample client, project, invoice, and line item data. It first clears existing records from related tables, then inserts the client and project data while mapping project names to their IDs. Finally, it processes invoice data by calculating totals and inserting invoices along with their corresponding line items.

```
import db from './db.js'
// Sample data for clients and their projects
const clients = [
 {
  name: 'Maya Chen',
  email: 'maya.chen@northstar-studio.com',
  phone: '+1 415 555 0194',
  company: 'Northstar Studio',
  address: '210 Market Street, Suite 804, San Francisco, CA 94105',
  notes: 'Prefers concise monthly status summaries.',
  projects: [
   {
    name: 'Brand Identity Refresh',
```

```
      description: 'Updated visual identity, design system foundations, and launch collateral.',
      status: 'completed',
      hourlyRateCents: 12500,
    },
    {
      name: 'Marketing Website Optimization',
      description: 'Landing page experiments, analytics cleanup, and conversion improvements.',
      status: 'active',
      hourlyRateCents: 11500,
    },
  ],
 },
 {
  name: 'Jordan Patel',
  email: 'jordan.patel@harborlegal.com',
  phone: '+1 212 555 0138',
  company: 'Harbor Legal Group',
  address: '88 Broad Street, Floor 12, New York, NY 10004',
  notes: 'Invoices route through operations before payment.',
  projects: [
   {
    name: 'Client Portal Planning',
    description: 'Discovery, technical planning, and implementation roadmap for a secure client portal.',
    status: 'completed',
```

```
      hourlyRateCents: 14000,
    },
    {
      name: 'Document Workflow Automation',
      description: 'Automation of intake forms, document generation, and internal review queues.',
      status: 'active',
      hourlyRateCents: 15000,
    },
  ],
 },
 {
  name: 'Elena Rodriguez',
  email: 'elena@brightpathfitness.co',
  phone: '+1 512 555 0177',
  company: 'BrightPath Fitness',
  address: '450 South Congress Avenue, Austin, TX 78704',
  notes: 'Seasonal campaigns align with class launches.',
  projects: [
   {
    name: 'Membership Dashboard',
    description: 'Member retention dashboard and reporting views for studio managers.',
    status: 'active',
    hourlyRateCents: 10500,
   },
   {
```

```
      name: 'Spring Campaign Assets',
      description: 'Email templates, ad creative, and performance reporting for spring promotions.',
      status: 'completed',
      hourlyRateCents: 9500,
    },
  ],
},
{
  name: 'Thomas Wright',
  email: 'thomas.wright@greenridgehomes.com',
  phone: '+1 303 555 0122',
  company: 'Greenridge Homes',
  address: '1720 Wazee Street, Denver, CO 80202',
  notes: 'Requests itemized invoices by project phase.',
  projects: [
    {
      name: 'Property Listing Platform',
      description: 'Custom listing pages, lead capture, and broker administration tools.',
      status: 'active',
      hourlyRateCents: 13000,
    },
    {
      name: 'Sales Collateral System',
      description: 'Reusable templates and content workflow for neighborhood sales materials.',
      status: 'completed',
```

```
      hourlyRateCents: 11000,
    },
  ],
 },
]
// Sample invoice data, each linked to a project by name
const invoiceData = [
 {
  projectName: 'Brand Identity Refresh',
  invoiceNumber: 'INV-2026-001',
  status: 'paid',
  issueDate: '2026-01-08',
  dueDate: '2026-02-07',
  sentAt: '2026-01-08T16:15:00.000Z',
  paidAt: '2026-01-24T18:40:00.000Z',
  notes: 'Final identity package and handoff.',
  items: [
   ['Discovery workshop and stakeholder interviews', 6, 12500],
   ['Logo refinement and visual identity exploration', 14, 12500],
   ['Brand guideline documentation', 8, 12500],
  ],
 },
 {
  projectName: 'Brand Identity Refresh',
  invoiceNumber: 'INV-2026-002',
  status: 'void',
```

```
  issueDate: '2026-02-03',
  dueDate: '2026-03-05',
  sentAt: '2026-02-03T15:30:00.000Z',
  paidAt: null,
  notes: 'Voided and replaced after scope correction.',
  items: [
    ['Unused rush production estimate', 5, 12500],
    ['Draft collateral resizing', 3, 12500],
  ],
},
{
  projectName: 'Marketing Website Optimization',
  invoiceNumber: 'INV-2026-003',
  status: 'sent',
  issueDate: '2026-06-14',
  dueDate: '2026-07-14',
  sentAt: '2026-06-14T13:05:00.000Z',
  paidAt: null,
  notes: 'June optimization sprint.',
  items: [
    ['Landing page copy and layout testing', 9, 11500],
    ['Analytics event cleanup', 4, 11500],
    ['Conversion report preparation', 2, 11500],
  ],
},
{
```

```
  projectName: 'Client Portal Planning',
  invoiceNumber: 'INV-2026-004',
  status: 'paid',
  issueDate: '2026-01-22',
  dueDate: '2026-02-21',
  sentAt: '2026-01-22T14:00:00.000Z',
  paidAt: '2026-02-12T20:10:00.000Z',
  notes: 'Discovery phase completed.',
  items: [
    ['Security requirements workshop', 5, 14000],
    ['Architecture planning and estimates', 10, 14000],
    ['Roadmap presentation', 3, 14000],
  ],
},
{
  projectName: 'Document Workflow Automation',
  invoiceNumber: 'INV-2026-005',
  status: 'overdue',
  issueDate: '2026-03-17',
  dueDate: '2026-04-16',
  sentAt: '2026-03-17T17:20:00.000Z',
  paidAt: null,
  notes: 'Awaiting operations approval.',
  items: [
    ['Form intake mapping', 7, 15000],
    ['Automation prototype build', 12, 15000],
```

```
    ['Review queue configuration', 6, 15000],
    ['Internal demo and revisions', 2, 15000],
  ],
},
{
  projectName: 'Document Workflow Automation',
  invoiceNumber: 'INV-2026-006',
  status: 'draft',
  issueDate: '2026-06-26',
  dueDate: '2026-07-26',
  sentAt: null,
  paidAt: null,
  notes: 'Draft for current implementation work.',
  items: [
    ['Template generation rules', 8, 15000],
    ['Reviewer notification workflow', 5, 15000],
    ['QA pass and bug fixes', 4, 15000],
  ],
},
{
  projectName: 'Membership Dashboard',
  invoiceNumber: 'INV-2026-007',
  status: 'sent',
  issueDate: '2026-05-29',
  dueDate: '2026-06-28',
  sentAt: '2026-05-29T12:45:00.000Z',
```

```
  paidAt: null,
  notes: 'Dashboard build milestone.',
  items: [
    ['Retention metric modeling', 6, 10500],
    ['Manager dashboard implementation', 15, 10500],
    ['Data export view', 4, 10500],
  ],
},
{
  projectName: 'Membership Dashboard',
  invoiceNumber: 'INV-2026-008',
  status: 'draft',
  issueDate: '2026-06-30',
  dueDate: '2026-07-30',
  sentAt: null,
  paidAt: null,
  notes: 'Draft for reporting refinements.',
  items: [
    ['Weekly cohort chart refinements', 5, 10500],
    ['Studio comparison filters', 6, 10500],
  ],
},
{
  projectName: 'Spring Campaign Assets',
  invoiceNumber: 'INV-2026-009',
  status: 'paid',
```

```
  issueDate: '2026-02-19',
  dueDate: '2026-03-21',
  sentAt: '2026-02-19T18:00:00.000Z',
  paidAt: '2026-03-08T16:35:00.000Z',
  notes: 'Campaign asset package.',
  items: [
    ['Email template design', 6, 9500],
    ['Paid social ad variants', 8, 9500],
    ['Campaign performance summary', 3, 9500],
  ],
},
{
  projectName: 'Property Listing Platform',
  invoiceNumber: 'INV-2026-010',
  status: 'overdue',
  issueDate: '2026-04-10',
  dueDate: '2026-05-10',
  sentAt: '2026-04-10T11:15:00.000Z',
  paidAt: null,
  notes: 'Listing platform implementation milestone.',
  items: [
    ['Listing detail page build', 11, 13000],
    ['Lead capture integration', 7, 13000],
    ['Broker admin permissions', 5, 13000],
  ],
},
```

```
{
  projectName: 'Property Listing Platform',
  invoiceNumber: 'INV-2026-011',
  status: 'sent',
  issueDate: '2026-06-05',
  dueDate: '2026-07-05',
  sentAt: '2026-06-05T14:25:00.000Z',
  paidAt: null,
  notes: 'Map and search improvements.',
  items: [
    ['Search filter improvements', 8, 13000],
    ['Map marker clustering', 6, 13000],
    ['Mobile listing QA', 4, 13000],
  ],
},
{
  projectName: 'Sales Collateral System',
  invoiceNumber: 'INV-2026-012',
  status: 'paid',
  issueDate: '2026-03-04',
  dueDate: '2026-04-03',
  sentAt: '2026-03-04T10:50:00.000Z',
  paidAt: '2026-03-29T19:05:00.000Z',
  notes: 'Template system delivery.',
  items: [
    ['Template component library', 9, 11000],
```

```
      ['Neighborhood flyer layouts', 7, 11000],
      ['Content publishing documentation', 3, 11000],
    ],
  },
]
// Prepared statements for database insertion
const insertClient = db.prepare(`
  INSERT INTO clients (name, email, phone, company, address, notes)
  VALUES (@name, @email, @phone, @company, @address, @notes)
`)
const insertProject = db.prepare(`
  INSERT INTO projects (client_id, name, description, status, hourly_rate_cents)
  VALUES (@clientId, @name, @description, @status, @hourlyRateCents)
`)
const insertInvoice = db.prepare(`
  INSERT INTO invoices (
    project_id,
    invoice_number,
    status,
    issue_date,
    due_date,
    currency,
    subtotal_cents,
    total_cents,
    paid_cents,
    sent_at,
```

```
    paid_at,
    notes
  )
  VALUES (
    @projectId,
    @invoiceNumber,
    @status,
    @issueDate,
    @dueDate,
    @currency,
    @subtotalCents,
    @totalCents,
    @paidCents,
    @sentAt,
    @paidAt,
    @notes
  )
`)
const insertLineItem = db.prepare(`
  INSERT INTO invoice_line_items (
    invoice_id,
    position,
    description,
    quantity,
    unit_price_cents,
    line_total_cents
```

```
  )
  VALUES (
    @invoiceId,
    @position,
    @description,
    @quantity,
    @unitPriceCents,
    @lineTotalCents
  )
`)
// Transaction ensures all inserts succeed or fail together
const seed = db.transaction(() => {
  // Clear existing data in reverse order of foreign key dependencies
  db.exec(`
    DELETE FROM invoice_line_items;
    DELETE FROM invoices;
    DELETE FROM projects;
    DELETE FROM clients;
  `)
  const projectIdsByName = new Map()
  // Insert clients and their projects, storing project IDs by name for invoice linking
  for (const client of clients) {
    const clientId = insertClient.run(client).lastInsertRowid
    for (const project of client.projects) {
      const projectId = insertProject.run({
        clientId,
```

```
    name: project.name,
    description: project.description,
    status: project.status,
    hourlyRateCents: project.hourlyRateCents,
  }).lastInsertRowid
  projectIdsByName.set(project.name, projectId)
 }
}
// Process each invoice, calculate totals, and insert invoice with line items
for (const invoice of invoiceData) {
 const projectId = projectIdsByName.get(invoice.projectName)
 if (!projectId) {
  throw new Error(`Missing project for invoice ${invoice.invoiceNumber}`)
 }
 // Calculate line item totals and overall invoice total
 const lineItems = invoice.items.map(([description, quantity, unitPriceCents], index) => ({
  position: index,
  description,
  quantity,
  unitPriceCents,
  lineTotalCents: Math.round(quantity * unitPriceCents),
 }))
 const totalCents = lineItems.reduce((total, item) => total + item.lineTotalCents, 0)
 const paidCents = invoice.status === 'paid' ? totalCents : 0
 const invoiceId = insertInvoice.run({
  projectId,
```

```
    invoiceNumber: invoice.invoiceNumber,
    status: invoice.status,
    issueDate: invoice.issueDate,
    dueDate: invoice.dueDate,
    currency: 'USD',
    subtotalCents: totalCents,
    totalCents,
    paidCents,
    sentAt: invoice.sentAt,
    paidAt: invoice.paidAt,
    notes: invoice.notes,
  }).lastInsertRowid
  // Insert each line item for this invoice
  for (const item of lineItems) {
    insertLineItem.run({
      invoiceId,
      ...item,
    })
  }
 }
})
seed()
```

services.js

Found in folder: **/src/lib**

This code provides utility functions and database operations for managing clients, projects, and invoices. It includes validation and normalization functions for data inputs, as well as

CRUD operations that interact with a SQLite database. The functions handle business logic like calculating totals, ensuring data integrity, and generating summary reports.

```
import 'server-only'

import db from './db.js'

// Valid status values for projects and invoices

const PROJECT_STATUSES = new Set(['active', 'paused', 'completed', 'archived'])

const INVOICE_STATUSES = new Set(['draft', 'sent', 'paid', 'overdue', 'void'])

// Returns current date/time in ISO format

function nowIso() {

  return new Date().toISOString()

}

// Returns first day of the month for a given date

function monthStart(date) {

  return new Date(Date.UTC(date.getUTCFullYear(), date.getUTCMonth(), 1))

}

// Adds specified number of months to a date

function addMonths(date, count) {

  return new Date(Date.UTC(date.getUTCFullYear(), date.getUTCMonth() + count, 1))

}

// Formats date as YYYY-MM-DD string

function formatDateOnly(date) {

  return date.toISOString().slice(0, 10)

}

// Formats date as "MMM YYYY" (e.g., "Jan 2024")

function formatMonthLabel(date) {

  return new Intl.DateTimeFormat('en-US', {
```

```
    month: 'short',
    year: 'numeric',
    timeZone: 'UTC',
  }).format(date)
}
// Converts ID to positive integer, throws if invalid
function normalizeId(id, label = 'id') {
  const value = Number(id)
  if (!Number.isInteger(value) || value <= 0) {
    throw new Error(`Invalid ${label}: expected a positive integer`)
  }
  return value
}
// Converts value to trimmed string or null if empty/undefined
function normalizeOptionalText(value) {
  if (value === undefined || value === null) {
    return null
  }
  const text = String(value).trim()
  return text === '' ? null : text
}
// Requires non-empty text value
function requireText(value, label) {
  const text = normalizeOptionalText(value)
  if (!text) {
    throw new Error(`${label} is required`)
```

```
  }
  return text
}
// Validates money amount in cents
function normalizeMoney(value, label, { allowNull = false } = {}) {
  if (value === undefined || value === null || value === '') {
    if (allowNull) {
      return null
    }
    throw new Error(`${label} is required`)
  }
  const cents = Number(value)
  if (!Number.isInteger(cents) || cents < 0) {
    throw new Error(`${label} must be a non-negative integer number of cents`)
  }
  return cents
}
// Validates quantity is a positive finite number
function normalizeQuantity(value, label = 'Quantity') {
  const quantity = Number(value)
  if (!Number.isFinite(quantity) || quantity <= 0) {
    throw new Error(`${label} must be greater than zero`)
  }
  return quantity
}
// Validates currency code (defaults to USD)
```

```
function normalizeCurrency(value) {
  const currency = value === undefined || value === null || value === ''
    ? 'USD'
    : String(value).trim().toUpperCase()
  if (!/^[A-Z]{3}$/.test(currency)) {
    throw new Error('Currency must be a 3-letter ISO code')
  }
  return currency
}
// Validates project status (defaults to 'active')
function normalizeProjectStatus(value) {
  const status = value === undefined || value === null || value === ''
    ? 'active'
    : String(value).trim()
  if (!PROJECT_STATUSES.has(status)) {
    throw new Error(`Invalid project status: ${status}`)
  }
  return status
}
// Validates invoice status (defaults to 'draft')
function normalizeInvoiceStatus(value) {
  const status = value === undefined || value === null || value === ''
    ? 'draft'
    : String(value).trim()
  if (!INVOICE_STATUSES.has(status)) {
    throw new Error(`Invalid invoice status: ${status}`)
```

```
  }
  return status
}
// Validates and normalizes invoice line items array
function normalizeLineItems(lineItems) {
  if (!Array.isArray(lineItems) || lineItems.length === 0) {
    throw new Error('At least one invoice line item is required')
  }
  return lineItems.map((item, index) => {
    const quantity = normalizeQuantity(item.quantity, `Line item ${index + 1} quantity`)
    const unitPriceCents = normalizeMoney(
      item.unitPriceCents,
      `Line item ${index + 1} unit price`,
    )
    return {
      position: index,
      description: requireText(item.description, `Line item ${index + 1} description`),
      quantity,
      unitPriceCents,
      lineTotalCents: Math.round(quantity * unitPriceCents),
    }
  })
}
// Validates and normalizes complete invoice data
function normalizeInvoiceData(data) {
  if (!data || typeof data !== 'object') {
```

```
  throw new Error('Invoice data is required')
}
const lineItems = normalizeLineItems(data.lineItems)
const totalCents = lineItems.reduce((sum, item) => sum + item.lineTotalCents, 0)
const paidCents = normalizeMoney(data.paidCents ?? 0, 'Paid amount')
const status = normalizeInvoiceStatus(data.status)
// Business rule validation
if (paidCents > totalCents) {
  throw new Error('Paid amount cannot exceed invoice total')
}
if (status === 'paid' && paidCents !== totalCents) {
  throw new Error('Paid invoices must have paid amount equal to total')
}
if (status !== 'paid' && data.paidAt) {
  throw new Error('Only paid invoices can have a paid date')
}
return {
  projectId: normalizeId(data.projectId, 'project id'),
  invoiceNumber: requireText(data.invoiceNumber, 'Invoice number'),
  status,
  issueDate: requireText(data.issueDate, 'Issue date'),
  dueDate: normalizeOptionalText(data.dueDate),
  currency: normalizeCurrency(data.currency),
  subtotalCents: totalCents,
  totalCents,
  paidCents,
```

```
    sentAt: normalizeOptionalText(data.sentAt),
    paidAt: normalizeOptionalText(data.paidAt),
    notes: normalizeOptionalText(data.notes),
    lineItems,
  }
}
// Maps database row to client object
function mapClient(row) {
  if (!row) {
    return null
  }
  return {
    id: row.id,
    name: row.name,
    email: row.email,
    phone: row.phone,
    company: row.company,
    address: row.address,
    notes: row.notes,
    createdAt: row.created_at,
    updatedAt: row.updated_at,
  }
}
// Maps database row to project object
function mapProject(row) {
  if (!row) {
```

```
    return null
  }
  return {
    id: row.id,
    clientId: row.client_id,
    name: row.name,
    description: row.description,
    status: row.status,
    hourlyRateCents: row.hourly_rate_cents,
    createdAt: row.created_at,
    updatedAt: row.updated_at,
  }
}
// Maps database row to invoice object
function mapInvoice(row) {
  if (!row) {
    return null
  }
  return {
    id: row.id,
    projectId: row.project_id,
    invoiceNumber: row.invoice_number,
    status: row.status,
    issueDate: row.issue_date,
    dueDate: row.due_date,
    currency: row.currency,
```

```
    subtotalCents: row.subtotal_cents,
    totalCents: row.total_cents,
    paidCents: row.paid_cents,
    sentAt: row.sent_at,
    paidAt: row.paid_at,
    notes: row.notes,
    createdAt: row.created_at,
    updatedAt: row.updated_at,
  }
}
// Maps database row to line item object
function mapLineItem(row) {
  return {
    id: row.id,
    invoiceId: row.invoice_id,
    position: row.position,
    description: row.description,
    quantity: row.quantity,
    unitPriceCents: row.unit_price_cents,
    lineTotalCents: row.line_total_cents,
    createdAt: row.created_at,
    updatedAt: row.updated_at,
  }
}
// Throws error if client doesn't exist
function ensureClientExists(id) {
```

```
  const client = db.prepare('SELECT id FROM clients WHERE id = ?').get(id)
  if (!client) {
    throw new Error(`Client ${id} does not exist`)
  }
}
// Throws error if project doesn't exist
function ensureProjectExists(id) {
  const project = db.prepare('SELECT id FROM projects WHERE id = ?').get(id)
  if (!project) {
    throw new Error(`Project ${id} does not exist`)
  }
}
// Throws error if invoice doesn't exist
function ensureInvoiceExists(id) {
  const invoice = db.prepare('SELECT id FROM invoices WHERE id = ?').get(id)
  if (!invoice) {
    throw new Error(`Invoice ${id} does not exist`)
  }
}
// Returns all clients with active project count and unpaid amount
export function getClients() {
  const rows = db.prepare(`
    SELECT
      c.*,
      COUNT(DISTINCT CASE WHEN p.status = 'active' THEN p.id END) AS
active_project_count,
```

```
    COALESCE(SUM(
      CASE
        WHEN i.status != 'void' AND i.paid_cents < i.total_cents
          THEN i.total_cents - i.paid_cents
        ELSE 0
      END
    ), 0) AS total_unpaid_cents
   FROM clients c
   LEFT JOIN projects p ON p.client_id = c.id
   LEFT JOIN invoices i ON i.project_id = p.id
   GROUP BY c.id
   ORDER BY c.name COLLATE NOCASE ASC
  `).all()
  return rows.map((row) => ({
   ...mapClient(row),
   activeProjectCount: row.active_project_count,
   totalUnpaidCents: row.total_unpaid_cents,
  }))
}
// Returns single client by ID with project count and unpaid amount
export function getClientById(id) {
  const clientId = normalizeId(id, 'client id')
  const row = db.prepare(`
   SELECT
    c.*,
    COUNT(DISTINCT p.id) AS project_count,
```

```
    COALESCE(SUM(
      CASE
        WHEN i.status != 'void' AND i.paid_cents < i.total_cents
          THEN i.total_cents - i.paid_cents
        ELSE 0
      END
    ), 0) AS total_unpaid_cents
   FROM clients c
   LEFT JOIN projects p ON p.client_id = c.id
   LEFT JOIN invoices i ON i.project_id = p.id
   WHERE c.id = ?
   GROUP BY c.id
  `).get(clientId)
  if (!row) {
   return null
  }
  return {
   ...mapClient(row),
   projectCount: row.project_count,
   totalUnpaidCents: row.total_unpaid_cents,
  }
}
// Creates new client record
export function createClient(data) {
  if (!data || typeof data !== 'object') {
    throw new Error('Client data is required')
```

```
  }
  const result = db.prepare(`
    INSERT INTO clients (name, email, phone, company, address, notes)
    VALUES (@name, @email, @phone, @company, @address, @notes)
  `).run({
    name: requireText(data.name, 'Client name'),
    email: normalizeOptionalText(data.email),
    phone: normalizeOptionalText(data.phone),
    company: normalizeOptionalText(data.company),
    address: normalizeOptionalText(data.address),
    notes: normalizeOptionalText(data.notes),
  })
  return getClientById(result.lastInsertRowid)
}
// Updates existing client record
export function updateClient(id, data) {
  const clientId = normalizeId(id, 'client id')
  const existing = mapClient(db.prepare('SELECT * FROM clients WHERE id =
?').get(clientId))
  if (!existing) {
    throw new Error(`Client ${clientId} does not exist`)
  }
  db.prepare(`
    UPDATE clients
    SET
      name = @name,
```

```
    email = @email,
    phone = @phone,
    company = @company,
    address = @address,
    notes = @notes,
    updated_at = @updatedAt
  WHERE id = @id
  `).run({
  id: clientId,
  name: data?.name === undefined ? existing.name : requireText(data.name, 'Client name'),
  email: data?.email === undefined ? existing.email : normalizeOptionalText(data.email),
  phone: data?.phone === undefined ? existing.phone : normalizeOptionalText(data.phone),
  company: data?.company === undefined ? existing.company : normalizeOptionalText(data.company),
  address: data?.address === undefined ? existing.address : normalizeOptionalText(data.address),
  notes: data?.notes === undefined ? existing.notes : normalizeOptionalText(data.notes),
  updatedAt: nowIso(),
 })
 return getClientById(clientId)
}
// Deletes client if they have no projects
export function deleteClient(id) {
 const clientId = normalizeId(id, 'client id')
 ensureClientExists(clientId)
```

```
  const projectCount = db.prepare('SELECT COUNT(*) AS count FROM projects WHERE client_id = ?')
    .get(clientId).count
  if (projectCount > 0) {
    throw new Error(`Cannot delete client ${clientId} because it has projects`)
  }
  db.prepare('DELETE FROM clients WHERE id = ?').run(clientId)
}
// Returns projects with optional filtering
export function getProjects(filters = {}) {
  const where = []
  const params = {}
  if (filters.clientId !== undefined && filters.clientId !== null && filters.clientId !== '') {
    params.clientId = normalizeId(filters.clientId, 'client id')
    where.push('p.client_id = @clientId')
  }
  if (filters.status !== undefined && filters.status !== null && filters.status !== '') {
    params.status = normalizeProjectStatus(filters.status)
    where.push('p.status = @status')
  }
  const rows = db.prepare(`
    SELECT
      p.*,
      c.name AS client_name,
      COALESCE(SUM(
        CASE
```

```
      WHEN i.status != 'void' THEN i.total_cents
      ELSE 0
    END
  ), 0) AS total_invoiced_cents
  FROM projects p
  INNER JOIN clients c ON c.id = p.client_id
  LEFT JOIN invoices i ON i.project_id = p.id
  ${where.length > 0 ? `WHERE ${where.join(' AND ')}` : ''}
  GROUP BY p.id
  ORDER BY p.name COLLATE NOCASE ASC
 `).all(params)
 return rows.map((row) => ({
  ...mapProject(row),
  clientName: row.client_name,
  totalInvoicedCents: row.total_invoiced_cents,
 }))
}
// Returns single project by ID with client details and invoice summary
export function getProjectById(id) {
 const projectId = normalizeId(id, 'project id')
 const row = db.prepare(`
  SELECT
   p.*,
   c.name AS client_name,
   c.email AS client_email,
   c.phone AS client_phone,
```

```
    c.company AS client_company,
    c.address AS client_address,
    c.notes AS client_notes,
    COUNT(i.id) AS invoice_count,
    COALESCE(SUM(i.total_cents), 0) AS invoice_total_cents,
    COALESCE(SUM(i.paid_cents), 0) AS invoice_paid_cents,
    COALESCE(SUM(
      CASE
        WHEN i.status != 'void' AND i.paid_cents < i.total_cents
          THEN i.total_cents - i.paid_cents
        ELSE 0
      END
    ), 0) AS invoice_unpaid_cents
  FROM projects p
  INNER JOIN clients c ON c.id = p.client_id
  LEFT JOIN invoices i ON i.project_id = p.id
  WHERE p.id = ?
  GROUP BY p.id
 `).get(projectId)
if (!row) {
  return null
}
return {
  ...mapProject(row),
  client: {
    id: row.client_id,
```

```
    name: row.client_name,
    email: row.client_email,
    phone: row.client_phone,
    company: row.client_company,
    address: row.client_address,
    notes: row.client_notes,
   },
   invoiceSummary: {
    count: row.invoice_count,
    totalCents: row.invoice_total_cents,
    paidCents: row.invoice_paid_cents,
    unpaidCents: row.invoice_unpaid_cents,
   },
  }
}
// Creates new project record
export function createProject(data) {
 if (!data || typeof data !== 'object') {
  throw new Error('Project data is required')
 }
 const clientId = normalizeId(data.clientId, 'client id')
 ensureClientExists(clientId)
 const result = db.prepare(`
  INSERT INTO projects (client_id, name, description, status, hourly_rate_cents)
  VALUES (@clientId, @name, @description, @status, @hourlyRateCents)
 `).run({
```

```
    clientId,
    name: requireText(data.name, 'Project name'),
    description: normalizeOptionalText(data.description),
    status: normalizeProjectStatus(data.status),
    hourlyRateCents: normalizeMoney(data.hourlyRateCents, 'Hourly rate', { allowNull: true }),
  })
  return getProjectById(result.lastInsertRowid)
}
// Updates existing project record
export function updateProject(id, data) {
  const projectId = normalizeId(id, 'project id')
  const existing = mapProject(db.prepare('SELECT * FROM projects WHERE id = ?').get(projectId))
  if (!existing) {
    throw new Error(`Project ${projectId} does not exist`)
  }
  const clientId = data?.clientId === undefined
    ? existing.clientId
    : normalizeId(data.clientId, 'client id')
  ensureClientExists(clientId)
  db.prepare(`
    UPDATE projects
    SET
      client_id = @clientId,
      name = @name,
      description = @description,
```

```
    status = @status,
    hourly_rate_cents = @hourlyRateCents,
    updated_at = @updatedAt
  WHERE id = @id
 `).run({
  id: projectId,
  clientId,
  name: data?.name === undefined ? existing.name : requireText(data.name, 'Project name'),
  description: data?.description === undefined
    ? existing.description
    : normalizeOptionalText(data.description),
  status: data?.status === undefined ? existing.status : normalizeProjectStatus(data.status),
  hourlyRateCents: data?.hourlyRateCents === undefined
    ? existing.hourlyRateCents
    : normalizeMoney(data.hourlyRateCents, 'Hourly rate', { allowNull: true }),
  updatedAt: nowIso(),
 })
 return getProjectById(projectId)
}
// Deletes project if it has no invoices
export function deleteProject(id) {
 const projectId = normalizeId(id, 'project id')
 ensureProjectExists(projectId)
 const invoiceCount = db.prepare('SELECT COUNT(*) AS count FROM invoices WHERE project_id = ?')
```

```
    .get(projectId).count
  if (invoiceCount > 0) {
    throw new Error(` Cannot delete project ${projectId} because it has invoices`)
  }
  db.prepare('DELETE FROM projects WHERE id = ?').run(projectId)
}
// Returns dashboard summary statistics
export function getDashboardSummary() {
  const currentMonthStart = monthStart(new Date())
  const nextMonthStart = addMonths(currentMonthStart, 1)
  const row = db.prepare(`
    SELECT
      COALESCE(SUM(
        CASE
          WHEN i.status IN ('sent', 'overdue') THEN i.total_cents - i.paid_cents
          ELSE 0
        END
      ), 0) AS total_outstanding_cents,
      COALESCE(SUM(
        CASE
          WHEN i.status = 'paid'
            AND i.paid_at >= @currentMonthStart
            AND i.paid_at < @nextMonthStart
            THEN i.paid_cents
          ELSE 0
        END
```

```
    ), 0) AS total_paid_this_month_cents,
    COUNT(CASE WHEN i.status = 'overdue' THEN 1 END) AS overdue_count
  FROM invoices i
 `).get({
  currentMonthStart: currentMonthStart.toISOString(),
  nextMonthStart: nextMonthStart.toISOString(),
 })
 const topClient = db.prepare(`
  SELECT c.name
  FROM clients c
  INNER JOIN projects p ON p.client_id = c.id
  INNER JOIN invoices i ON i.project_id = p.id
  GROUP BY c.id
  HAVING SUM(i.paid_cents) > 0
  ORDER BY SUM(i.paid_cents) DESC, c.name COLLATE NOCASE ASC
  LIMIT 1
 `).get()
 return {
  totalOutstandingCents: row.total_outstanding_cents,
  totalPaidThisMonthCents: row.total_paid_this_month_cents,
  overdueCount: row.overdue_count,
  topClientByRevenue: topClient?.name ?? null,
 }
}
// Returns revenue data for last 6 months
export function getMonthlyRevenue() {
```

```
const currentMonthStart = monthStart(new Date())
const months = Array.from({ length: 6 }, (_, index) => {
 const start = addMonths(currentMonthStart, index - 5)
 const end = addMonths(start, 1)
 return {
  month: formatMonthLabel(start),
  start: formatDateOnly(start),
  end: formatDateOnly(end),
 }
})
const totalsByMonth = new Map(db.prepare(`
 SELECT
  strftime('%Y-%m', issue_date) AS month_key,
  COALESCE(SUM(total_cents), 0) AS total_cents
 FROM invoices
 WHERE issue_date >= @startDate
  AND issue_date < @endDate
 GROUP BY month_key
`).all({
 startDate: months[0].start,
 endDate: months[months.length - 1].end,
}).map((row) => [row.month_key, row.total_cents]))
return months.map((month) => ({
 month: month.month,
 totalCents: totalsByMonth.get(month.start.slice(0, 7)) ?? 0,
}))
```

```
}
// Returns all overdue invoices with client/project details
export function getOverdueInvoices() {
  const rows = db.prepare(`
    SELECT
      i.*,
      p.name AS project_name,
      c.id AS client_id,
      c.name AS client_name
    FROM invoices i
    INNER JOIN projects p ON p.id = i.project_id
    INNER JOIN clients c ON c.id = p.client_id
    WHERE i.status = 'overdue'
    ORDER BY i.due_date ASC, i.id ASC
  `).all()
  return rows.map((row) => ({
    ...mapInvoice(row),
    clientId: row.client_id,
    clientName: row.client_name,
    projectName: row.project_name,
  }))
}
// Returns invoices with optional filtering
export function getInvoices(filters = {}) {
  const where = []
  const params = {}
```

```
if (filters.status !== undefined && filters.status !== null && filters.status !== '') {
  params.status = normalizeInvoiceStatus(filters.status)
  where.push('i.status = @status')
}
if (filters.clientId !== undefined && filters.clientId !== null && filters.clientId !== '') {
  params.clientId = normalizeId(filters.clientId, 'client id')
  where.push('p.client_id = @clientId')
}
if (filters.projectId !== undefined && filters.projectId !== null && filters.projectId !== '') {
  params.projectId = normalizeId(filters.projectId, 'project id')
  where.push('i.project_id = @projectId')
}
const rows = db.prepare(`
  SELECT
    i.*,
    p.name AS project_name,
    c.id AS client_id,
    c.name AS client_name
  FROM invoices i
  INNER JOIN projects p ON p.id = i.project_id
  INNER JOIN clients c ON c.id = p.client_id
  ${where.length > 0 ? `WHERE ${where.join(' AND ')}` : ''}
  ORDER BY i.issue_date DESC, i.id DESC
`).all(params)
return rows.map((row) => ({
  ...mapInvoice(row),
```

```
    projectName: row.project_name,
    clientId: row.client_id,
    clientName: row.client_name,
  }))
}
// Returns single invoice by ID with line items and related details
export function getInvoiceById(id) {
  const invoiceId = normalizeId(id, 'invoice id')
  const row = db.prepare(`
    SELECT
      i.*,
      p.name AS project_name,
      p.description AS project_description,
      p.status AS project_status,
      p.hourly_rate_cents AS project_hourly_rate_cents,
      c.id AS client_id,
      c.name AS client_name,
      c.email AS client_email,
      c.phone AS client_phone,
      c.company AS client_company,
      c.address AS client_address,
      c.notes AS client_notes
    FROM invoices i
    INNER JOIN projects p ON p.id = i.project_id
    INNER JOIN clients c ON c.id = p.client_id
    WHERE i.id = ?
```

```
  `).get(invoiceId)
  if (!row) {
    return null
  }
  const lineItems = db.prepare(`
    SELECT *
    FROM invoice_line_items
    WHERE invoice_id = ?
    ORDER BY position ASC
  `).all(invoiceId).map(mapLineItem)
  return {
    ...mapInvoice(row),
    lineItems,
    project: {
      id: row.project_id,
      name: row.project_name,
      description: row.project_description,
      status: row.project_status,
      hourlyRateCents: row.project_hourly_rate_cents,
    },
    client: {
      id: row.client_id,
      name: row.client_name,
      email: row.client_email,
      phone: row.client_phone,
      company: row.client_company,
```

```
    address: row.client_address,
    notes: row.client_notes,
  },
 }
}
// Transaction to create invoice with line items atomically
const insertInvoiceWithItems = db.transaction((data) => {
 ensureProjectExists(data.projectId)
 const result = db.prepare(`
  INSERT INTO invoices (
   project_id,
   invoice_number,
   status,
   issue_date,
   due_date,
   currency,
   subtotal_cents,
   total_cents,
   paid_cents,
   sent_at,
   paid_at,
   notes
  )
  VALUES (
   @projectId,
   @invoiceNumber,
```

```
    @status,
    @issueDate,
    @dueDate,
    @currency,
    @subtotalCents,
    @totalCents,
    @paidCents,
    @sentAt,
    @paidAt,
    @notes
  )
`).run(data)
const invoiceId = result.lastInsertRowid
const insertLineItem = db.prepare(`
  INSERT INTO invoice_line_items (
    invoice_id,
    position,
    description,
    quantity,
    unit_price_cents,
    line_total_cents
  )
  VALUES (
    @invoiceId,
    @position,
    @description,
```

```
    @quantity,
    @unitPriceCents,
    @lineTotalCents
  )
 `)
 for (const item of data.lineItems) {
  insertLineItem.run({
   invoiceId,
   ...item,
  })
 }
 return invoiceId
})
// Creates new invoice with validation
export function createInvoice(data) {
 const invoice = normalizeInvoiceData(data)
 try {
  const invoiceId = insertInvoiceWithItems(invoice)
  return getInvoiceById(invoiceId)
 } catch (error) {
  if (error.code === 'SQLITE_CONSTRAINT_UNIQUE') {
   throw new Error(`Invoice number ${invoice.invoiceNumber} already exists`)
  }
  throw error
 }
}
```

```
// Transaction to update invoice and replace line items atomically
const updateInvoiceWithItems = db.transaction((id, data) => {
  ensureInvoiceExists(id)
  ensureProjectExists(data.projectId)
  db.prepare(`
    UPDATE invoices
    SET
      project_id = @projectId,
      invoice_number = @invoiceNumber,
      status = @status,
      issue_date = @issueDate,
      due_date = @dueDate,
      currency = @currency,
      subtotal_cents = @subtotalCents,
      total_cents = @totalCents,
      paid_cents = @paidCents,
      sent_at = @sentAt,
      paid_at = @paidAt,
      notes = @notes,
      updated_at = @updatedAt
    WHERE id = @id
  `).run({
    id,
    updatedAt: nowIso(),
    ...data,
  })
```

```
 db.prepare('DELETE FROM invoice_line_items WHERE invoice_id = ?').run(id)
 const insertLineItem = db.prepare(`
  INSERT INTO invoice_line_items (
    invoice_id,
    position,
    description,
    quantity,
    unit_price_cents,
    line_total_cents
  )
  VALUES (
    @invoiceId,
    @position,
    @description,
    @quantity,
    @unitPriceCents,
    @lineTotalCents
  )
 `)
 for (const item of data.lineItems) {
  insertLineItem.run({
    invoiceId: id,
    ...item,
  })
 }
})
```

```
// Updates existing invoice with validation
export function updateInvoice(id, data) {
  const invoiceId = normalizeId(id, 'invoice id')
  const invoice = normalizeInvoiceData(data)
  try {
    updateInvoiceWithItems(invoiceId, invoice)
    return getInvoiceById(invoiceId)
  } catch (error) {
    if (error.code === 'SQLITE_CONSTRAINT_UNIQUE') {
      throw new Error(`Invoice number ${invoice.invoiceNumber} already exists`)
    }
    throw error
  }
}
// Deletes invoice record
export function deleteInvoice(id) {
  const invoiceId = normalizeId(id, 'invoice id')
  ensureInvoiceExists(invoiceId)
  db.prepare('DELETE FROM invoices WHERE id = ?').run(invoiceId)
}
```

services.test.js

Found in folder: **/src/lib**

This code sets up a test suite for client, invoice, project, and dashboard services. It includes helper functions for date manipulation and seeding a test database with sample data. Each test case verifies that the service functions correctly handle data operations like creation, retrieval, and deletion.

```
import { beforeEach, describe, expect, it } from 'vitest'
```

```
import db from './db.js'
import './migrate.js'
import {
  createClient,
  createInvoice,
  deleteClient,
  deleteInvoice,
  getDashboardSummary,
  getClientById,
  getClients,
  getInvoiceById,
  getInvoices,
  getMonthlyRevenue,
  getOverdueInvoices,
  getProjects,
} from './services.js'
// Helper functions for date calculations
function monthStart(date) {
  return new Date(Date.UTC(date.getUTCFullYear(), date.getUTCMonth(), 1))
}
function addMonths(date, count) {
  return new Date(Date.UTC(date.getUTCFullYear(), date.getUTCMonth() + count, 1))
}
function dateOnly(date) {
  return date.toISOString().slice(0, 10)
}
```

```
function monthLabel(date) {
  return new Intl.DateTimeFormat('en-US', {
    month: 'short',
    year: 'numeric',
    timeZone: 'UTC',
  }).format(date)
}
// Populates the database with test data before each test
function seedTestData() {
  // Clear existing data
  db.exec(`
    DELETE FROM invoice_line_items;
    DELETE FROM invoices;
    DELETE FROM projects;
    DELETE FROM clients;
  `)
  // Prepared statements for inserting test data
  const insertClient = db.prepare(`
    INSERT INTO clients (name, email, phone, company, address, notes)
    VALUES (@name, @email, @phone, @company, @address, @notes)
  `)
  const insertProject = db.prepare(`
    INSERT INTO projects (client_id, name, description, status, hourly_rate_cents)
    VALUES (@clientId, @name, @description, @status, @hourlyRateCents)
  `)
  const insertInvoice = db.prepare(`
```

```
INSERT INTO invoices (
  project_id,
  invoice_number,
  status,
  issue_date,
  due_date,
  currency,
  subtotal_cents,
  total_cents,
  paid_cents,
  sent_at,
  paid_at,
  notes
)
VALUES (
  @projectId,
  @invoiceNumber,
  @status,
  @issueDate,
  @dueDate,
  @currency,
  @subtotalCents,
  @totalCents,
  @paidCents,
  @sentAt,
  @paidAt,
```

```
    @notes
  )
`)
const insertLineItem = db.prepare(`
  INSERT INTO invoice_line_items (
    invoice_id,
    position,
    description,
    quantity,
    unit_price_cents,
    line_total_cents
  )
  VALUES (
    @invoiceId,
    @position,
    @description,
    @quantity,
    @unitPriceCents,
    @lineTotalCents
  )
`)
// Insert two test clients
const acmeClientId = insertClient.run({
  name: 'Acme Studio',
  email: 'billing@acme.test',
  phone: null,
```

```
  company: 'Acme',
  address: null,
  notes: 'Primary test client',
}).lastInsertRowid
const betaClientId = insertClient.run({
  name: 'Beta Labs',
  email: 'ops@beta.test',
  phone: null,
  company: 'Beta',
  address: null,
  notes: null,
}).lastInsertRowid
// Insert a project for Acme
const projectId = insertProject.run({
  clientId: acmeClientId,
  name: 'Website Build',
  description: 'Marketing site',
  status: 'active',
  hourlyRateCents: 10000,
}).lastInsertRowid
// Insert an invoice for the project
const invoiceId = insertInvoice.run({
  projectId,
  invoiceNumber: 'INV-TEST-001',
  status: 'sent',
  issueDate: '2026-06-01',
```

```
  dueDate: '2026-07-01',
  currency: 'USD',
  subtotalCents: 25000,
  totalCents: 25000,
  paidCents: 0,
  sentAt: '2026-06-01T12:00:00.000Z',
  paidAt: null,
  notes: 'Seed invoice',
}).lastInsertRowid
// Add line items to the invoice
insertLineItem.run({
  invoiceId,
  position: 0,
  description: 'Design',
  quantity: 1,
  unitPriceCents: 10000,
  lineTotalCents: 10000,
})
insertLineItem.run({
  invoiceId,
  position: 1,
  description: 'Development',
  quantity: 3,
  unitPriceCents: 5000,
  lineTotalCents: 15000,
})
```

```
  return {
    acmeClientId,
    betaClientId,
    projectId,
    invoiceId,
  }
}
// Helper to insert a project for testing
function insertProject(clientId, name) {
  return db.prepare(`
    INSERT INTO projects (client_id, name, description, status, hourly_rate_cents)
    VALUES (?, ?, ?, ?, ?)
  `).run(clientId, name, null, 'active', null).lastInsertRowid
}
// Helper to insert an invoice for testing
function insertInvoice({
  projectId,
  invoiceNumber,
  status,
  issueDate,
  dueDate = null,
  totalCents,
  paidCents = 0,
  paidAt = null,
}) {
  return db.prepare(`
```

```
INSERT INTO invoices (
  project_id,
  invoice_number,
  status,
  issue_date,
  due_date,
  currency,
  subtotal_cents,
  total_cents,
  paid_cents,
  sent_at,
  paid_at,
  notes
)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
`).run(
projectId,
invoiceNumber,
status,
issueDate,
dueDate,
'USD',
totalCents,
totalCents,
paidCents,
null,
```

```
    paidAt,
    null,
  ).lastInsertRowid
}
let fixture
// Reset test data before each test
beforeEach(() => {
  fixture = seedTestData()
})
describe('client services', () => {
  it('getClients() returns all clients', () => {
    const clients = getClients()
    expect(clients).toHaveLength(2)
    expect(clients.map((client) => client.name)).toEqual(['Acme Studio', 'Beta Labs'])
    expect(clients[0]).toMatchObject({
      activeProjectCount: 1,
      totalUnpaidCents: 25000,
    })
    expect(clients[1]).toMatchObject({
      activeProjectCount: 0,
      totalUnpaidCents: 0,
    })
  })
  it('getClients() counts active projects and unpaid balances', () => {
    // Add a paused project and additional invoices to test aggregation
    const pausedProjectId = db.prepare(`
```

```
  INSERT INTO projects (client_id, name, description, status, hourly_rate_cents)
  VALUES (?, ?, ?, ?, ?)
`).run(
  fixture.acmeClientId,
  'Retainer',
  'Paused work',
  'paused',
  12000,
).lastInsertRowid
db.prepare(`
  INSERT INTO invoices (
    project_id,
    invoice_number,
    status,
    issue_date,
    due_date,
    currency,
    subtotal_cents,
    total_cents,
    paid_cents,
    sent_at,
    paid_at,
    notes
  )
  VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
`).run(
```

```
  pausedProjectId,
  'INV-TEST-002',
  'sent',
  '2026-06-10',
  '2026-07-10',
  'USD',
  10000,
  10000,
  2500,
  '2026-06-10T12:00:00.000Z',
  null,
  null,
)
db.prepare(`
  INSERT INTO invoices (
    project_id,
    invoice_number,
    status,
    issue_date,
    due_date,
    currency,
    subtotal_cents,
    total_cents,
    paid_cents,
    sent_at,
    paid_at,
```

```
    notes
  )
  VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
 `).run(
  fixture.projectId,
  'INV-TEST-003',
  'void',
  '2026-06-11',
  '2026-07-11',
  'USD',
  50000,
  50000,
  0,
  null,
  null,
  null,
 )
 const [client] = getClients()
 expect(client).toMatchObject({
  id: fixture.acmeClientId,
  activeProjectCount: 1,
  totalUnpaidCents: 32500,
 })
})
it('getClientById() returns the correct client', () => {
 const client = getClientById(fixture.acmeClientId)
```

```
  expect(client).toMatchObject({
    id: fixture.acmeClientId,
    name: 'Acme Studio',
    projectCount: 1,
    totalUnpaidCents: 25000,
  })
})
it('createClient() inserts and returns a new client', () => {
  const client = createClient({
    name: 'Gamma Co',
    email: 'hello@gamma.test',
    phone: '555-0100',
    company: 'Gamma',
    address: '1 Test Way',
    notes: 'New client',
  })
  expect(client).toMatchObject({
    name: 'Gamma Co',
    email: 'hello@gamma.test',
    projectCount: 0,
    totalUnpaidCents: 0,
  })
  expect(getClients()).toHaveLength(3)
})
it('deleteClient() throws when projects exist', () => {
  expect(() => deleteClient(fixture.acmeClientId))
```

```
    .toThrow(`Cannot delete client ${fixture.acmeClientId} because it has projects`)
  })
})
describe('invoice services', () => {
  it('getInvoices() returns all invoices', () => {
    const invoices = getInvoices()
    expect(invoices).toHaveLength(1)
    expect(invoices[0]).toMatchObject({
      id: fixture.invoiceId,
      invoiceNumber: 'INV-TEST-001',
      clientName: 'Acme Studio',
      projectName: 'Website Build',
    })
  })
  it('getInvoiceById() returns the invoice with line items', () => {
    const invoice = getInvoiceById(fixture.invoiceId)
    expect(invoice).toMatchObject({
      id: fixture.invoiceId,
      invoiceNumber: 'INV-TEST-001',
      totalCents: 25000,
      client: {
        id: fixture.acmeClientId,
        name: 'Acme Studio',
      },
      project: {
        id: fixture.projectId,
```

```
    name: 'Website Build',
   },
  })
  expect(invoice.lineItems).toHaveLength(2)
  expect(invoice.lineItems.map((item) => item.lineTotalCents)).toEqual([10000, 15000])
})
it('createInvoice() calculates totals correctly from line items', () => {
  const invoice = createInvoice({
   projectId: fixture.projectId,
   invoiceNumber: 'INV-TEST-002',
   status: 'draft',
   issueDate: '2026-06-15',
   dueDate: '2026-07-15',
   currency: 'USD',
   lineItems: [
    {
     description: 'Strategy',
     quantity: 2,
     unitPriceCents: 12500,
    },
    {
     description: 'Implementation',
     quantity: 1.5,
     unitPriceCents: 20000,
    },
   ],
```

```
  })
  expect(invoice.subtotalCents).toBe(55000)
  expect(invoice.totalCents).toBe(55000)
  expect(invoice.paidCents).toBe(0)
  expect(invoice.lineItems.map((item) => item.lineTotalCents)).toEqual([25000, 30000])
 })
 it('deleteInvoice() cascades to line items', () => {
  deleteInvoice(fixture.invoiceId)
  const invoiceCount = db.prepare('SELECT COUNT(*) AS count FROM invoices WHERE id
= ?')
    .get(fixture.invoiceId).count
  const lineItemCount = db.prepare(`
    SELECT COUNT(*) AS count
    FROM invoice_line_items
    WHERE invoice_id = ?
  `).get(fixture.invoiceId).count
  expect(invoiceCount).toBe(0)
  expect(lineItemCount).toBe(0)
 })
})
describe('project services', () => {
 it('getProjects() returns projects with non-void invoiced totals', () => {
  // Add a void invoice to ensure it's excluded from totals
  db.prepare(`
    INSERT INTO invoices (
      project_id,
```

```
  invoice_number,
  status,
  issue_date,
  due_date,
  currency,
  subtotal_cents,
  total_cents,
  paid_cents,
  sent_at,
  paid_at,
  notes
)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
`).run(
fixture.projectId,
'INV-TEST-VOID',
'void',
'2026-06-20',
'2026-07-20',
'USD',
50000,
50000,
0,
null,
null,
null,
```

```
  )
  const projects = getProjects()
  expect(projects).toHaveLength(1)
  expect(projects[0]).toMatchObject({
   id: fixture.projectId,
   clientId: fixture.acmeClientId,
   clientName: 'Acme Studio',
   name: 'Website Build',
   totalInvoicedCents: 25000,
  })
 })
})
describe('dashboard services', () => {
 it('getDashboardSummary() returns outstanding totals, paid month total, overdue count, and top client', () => {
  const currentMonth = monthStart(new Date())
  const previousMonth = addMonths(currentMonth, -1)
  const betaProjectId = insertProject(fixture.betaClientId, 'Analytics Cleanup')
  // Insert test invoices for different scenarios
  insertInvoice({
   projectId: fixture.projectId,
   invoiceNumber: 'INV-DASH-OVERDUE-ACME',
   status: 'overdue',
   issueDate: dateOnly(previousMonth),
   dueDate: dateOnly(addMonths(currentMonth, -1)),
   totalCents: 20000,
```

```
  paidCents: 5000,
})
insertInvoice({
  projectId: betaProjectId,
  invoiceNumber: 'INV-DASH-PAID-BETA',
  status: 'paid',
  issueDate: dateOnly(currentMonth),
  dueDate: dateOnly(currentMonth),
  totalCents: 40000,
  paidCents: 40000,
  paidAt: currentMonth.toISOString(),
})
insertInvoice({
  projectId: fixture.projectId,
  invoiceNumber: 'INV-DASH-PAID-ACME-OLD',
  status: 'paid',
  issueDate: dateOnly(previousMonth),
  dueDate: dateOnly(previousMonth),
  totalCents: 50000,
  paidCents: 50000,
  paidAt: previousMonth.toISOString(),
})
expect(getDashboardSummary()).toEqual({
  totalOutstandingCents: 40000,
  totalPaidThisMonthCents: 40000,
  overdueCount: 1,
```

```
    topClientByRevenue: 'Acme Studio',
  })
})
it('getMonthlyRevenue() returns the last 6 months with zero-filled totals', () => {
  const currentMonth = monthStart(new Date())
  const fiveMonthsAgo = addMonths(currentMonth, -5)
  const twoMonthsAgo = addMonths(currentMonth, -2)
  const outsideWindow = addMonths(currentMonth, -6)
  // Insert invoices within and outside the 6-month window
  insertInvoice({
    projectId: fixture.projectId,
    invoiceNumber: 'INV-REV-FIVE-MONTHS-AGO',
    status: 'paid',
    issueDate: dateOnly(fiveMonthsAgo),
    dueDate: dateOnly(fiveMonthsAgo),
    totalCents: 30000,
    paidCents: 30000,
    paidAt: fiveMonthsAgo.toISOString(),
  })
  insertInvoice({
    projectId: fixture.projectId,
    invoiceNumber: 'INV-REV-TWO-MONTHS-AGO',
    status: 'sent',
    issueDate: dateOnly(twoMonthsAgo),
    dueDate: dateOnly(twoMonthsAgo),
    totalCents: 45000,
```

```
})
insertInvoice({
  projectId: fixture.projectId,
  invoiceNumber: 'INV-REV-OUTSIDE-WINDOW',
  status: 'paid',
  issueDate: dateOnly(outsideWindow),
  dueDate: dateOnly(outsideWindow),
  totalCents: 99000,
  paidCents: 99000,
  paidAt: outsideWindow.toISOString(),
})
// Build expected totals map including the seed invoice
const expectedTotals = new Map([
  [dateOnly(fiveMonthsAgo).slice(0, 7), 30000],
  [dateOnly(twoMonthsAgo).slice(0, 7), 45000],
])
const seedMonthKey = '2026-06'
expectedTotals.set(seedMonthKey, (expectedTotals.get(seedMonthKey) ?? 0) + 25000)
expect(getMonthlyRevenue()).toEqual(
  Array.from({ length: 6 }, (_, index) => {
    const month = addMonths(currentMonth, index - 5)
    const monthKey = dateOnly(month).slice(0, 7)
    return {
      month: monthLabel(month),
      totalCents: expectedTotals.get(monthKey) ?? 0,
    }
```

```
    }),
  )
})
it('getOverdueInvoices() returns overdue invoices ordered by due date', () => {
  const betaProjectId = insertProject(fixture.betaClientId, 'Reporting Setup')
  const laterInvoiceId = insertInvoice({
    projectId: fixture.projectId,
    invoiceNumber: 'INV-OVERDUE-LATER',
    status: 'overdue',
    issueDate: '2026-03-15',
    dueDate: '2026-04-15',
    totalCents: 18000,
  })
  const earlierInvoiceId = insertInvoice({
    projectId: betaProjectId,
    invoiceNumber: 'INV-OVERDUE-EARLIER',
    status: 'overdue',
    issueDate: '2026-02-01',
    dueDate: '2026-03-01',
    totalCents: 27000,
  })
  // Non-overdue invoice should be excluded
  insertInvoice({
    projectId: fixture.projectId,
    invoiceNumber: 'INV-NOT-OVERDUE',
    status: 'sent',
```

```
    issueDate: '2026-03-01',
    dueDate: '2026-03-31',
    totalCents: 12000,
  })
  expect(getOverdueInvoices()).toMatchObject([
    {
      id: earlierInvoiceId,
      clientName: 'Beta Labs',
      projectName: 'Reporting Setup',
      dueDate: '2026-03-01',
      totalCents: 27000,
    },
    {
      id: laterInvoiceId,
      clientName: 'Acme Studio',
      projectName: 'Website Build',
      dueDate: '2026-04-15',
      totalCents: 18000,
    },
  ])
 })
})
```